University of Macedonia

Department of Applied Informatics

# O(1) and CF Schedulers
# Simulation and Comparison

Bachelor's Thesis of

**Kefsenidis Paraskevas**

Supervisor

**Professor Manos Roumeliotis**


Greece Thessaloniki, July 2024
Copyright ©

# Dedication

*To my aunt, for encouraging me to write this thesis*

*And my father, for making me sign up for Computer Science*

# Acknowledgements

I would like to thank my professor Manos Roumeliotis, for giving me the chance to write this thesis and of course for all the help he provided for the development of the simulation program, as well as for all the advises he gave me. Then my aunt Efstathia Kefsenidoy, for making me change my mind and start writing a thesis. And lastly, my friend Giorgos Mamidakis for helping me find a suitable thesis topic for me.

# Περίληψη

Από τη δημιουργία του έως σήμερα, αναπτύχθηκαν πολλοί χρονοδρομολογητές για τον πυρήνα του Linux. Ξεκινώντας από το O(n), ο οποίος είναι πολύ απλός και πηγαίνοντας στον O(N), τον O(1) και φτάνοντας στον περίπλοκο, ως προς το σχεδιασμό του, CFS. Ωστόσο, δεν σταματούν εκεί, καθώς κάποιοι που απογοητεύτηκαν από τους προαναφερθέντες χρονοδρομολογητές ανέπτυξαν άλλους, όπως τον MuQSS και τον BFS. Όλοι αυτοί έχουν κάτι κοινό: αναπτύχθηκαν για να αντικαταστήσουν κάποιον άλλο χρονοδρομολογητή. Η ερώτηση που προκύπτει τώρα είναι: ποιες είναι οι διαφορές τους· Με το να συγκρίνει δύο χρονοδρομολογητές (ένα δείγμα από τον συνολικό πληθυσμό των χρονοδρομολογητών) διαφορετικής πολυπλοκότητας, αυτοί είναι οι O(1) και CFS, αυτή η πτυχιακή έχει σκοπό να απαντήσει σε αυτήν την ερώτηση. Αρχικά, θα εξηγήσω πώς λειτουργούν οι δύο χρονοδρομολογητές και πώς είναι σχεδιασμένοι, και στη συνέχεια, προσομοιώνοντας τους με ένα απλό πρόγραμμα που ανέπτυξα, θα συγκεντρώσω κάποια στατιστικά στοιχεία που θα αποδειχθούν χρήσιμα στη διαδικασία σύγκρισής τους. Τελικά, αυτή η πτυχιακή καταλήγει στο ότι οι μεγαλύτερες διαφορές μεταξύ των χρονοδρομολογητών μπορεί να αποδειχθούν το σημαντικότερο τους πλεονέκτημα, έναντι των άλλων, το οποίο σημαίνει ότι καθένας του είναι σχεδιασμένος για να λειτουργεί κατάλληλα σε διαφορετικά φορτία εργασίας.

**Λέξεις κλειδιά**: O(1), CFS, σύγκριση, χρονοδρομολογητής, λειτουργικό σύστημα, Linux

# Abstract

From its creation until today, many schedulers were developed for the Linux kernel. Starting from O(n), which is very simple and going to O(N), O(1) and reaching to the complicated in design CFS. Though, they do not stop, as people disappointed with the aforementioned schedulers, developed others, like MuQSS and BFS. All of these have something in common: they were developed to replace some other scheduler. The question that rises, now is, what are their differences? By comparing two schedulers (a sample of the whole scheduler population) of different complexity, those being O(1) and CFS, this thesis aims to answer this question. Firstly, I will explain how the two schedulers work and how are they designed and then by simulating them with a simple program I developed, I will gather some statistics which will prove useful in the process of comparing them. Finally, this thesis concludes, that the biggest differences between schedulers, can prove to be their most significant advantage against each other, which actually means, that each one is suitable for different workloads.

**Keywords**: O(1), CFS, scheduler, comparison, simulation, operating system, Linux

# Contents

# List of Figures

# List of Listings

# Chapter 1

# Introduction

## 1.1 Motivation

One of the main reasons I started this thesis, was because I wanted to get better acquainted with the kernel source code. I had previous experience with auditing some parts of the GLIBC source code (especially malloc internals), but my auditing skills were quite vague. What I am trying to say is that, I did not know where to look in order to understand what a part of the code would do and I relied upon other sources to help me understand.

I wanted this to change, to actually learn a way of how to audit such complex open source projects, without relying on external sources for help. At the time, I was learning about binary exploitation and ultimately wanted to start learning about kernel exploitation. So I gathered some papers and books, discussing about the kernel and how some parts work. I decided to start from the scheduler, as my professor and advisor, of this thesis, once noted, that it is the most fundamental part of an operating system. Alongside these sources, I started auditing the scheduler source code, until I could be able to find a productive way of auditing it.

Other than that, I soon discovered, that the companies maintaining the Linux kernel decided to replace a scheduler of constant time complexity, with one of logarithmic time complexity. This actually drew my attention and wanted to know the reason behind this decision. Not only this, but I also discovered, that there were many schedulers implemented for Linux. So after some time of not knowing what would an interesting topic, for this thesis, be, I finally came to a conclusion and decided to compare the O(1) and CF schedulers.

## 1.2 Objectives

The main objective of this thesis is first and foremost compare the O(1) and CF schedulers and then using the results of this comparison, try to understand, the reason for the existence of so many schedulers, for just one operating system.

## 1.3 Thesis Structure

This thesis is structured in the following way:

1. Chapter 2 provides the reader, with some general information of how the two schedulers work and how they are implemented. It also provides some explanations of the source code. You can skip this part if you are already familiar with these two schedulers

2. Chapter 3, explains the methodology I followed in order to get the results of the comparison

3. Chapter 4, discusses the results and tries to make some assumptions about their differences and their problems and

4. Chapter 5, finalizes the assumptions previously made and answers to this thesis' objectives, while also providing some ideas for future work.

# Chapter 2

# Theoretical Background

In this section I will explain the fundamentals of the O(1) and CF Schedulers[1]. I will describe how they keep track and how they interact with a task's[2] data and how each of them implements the CPU's runqueue. Finally I will explain how they interact, with the aforementioned type structures (runqueue and task), in order to perform their scheduling policies.

## 2.1 Basics of a task

Before I explain how the two schedulers work, I first have to explain two of the most fundamental terms used in a scheduler. These are context switching (which is discussed in the next section) and the task.

A task is an instance of a program, that is being executed [1]. The term explicitly specifies that it is "an instance". This means that, multiple tasks, of the same program may exist at the same time. Also note here, that tasks, in contrast to a program, have a lifetime, which starts with the program's execution and ends with a program's termination.

A task's main job, is to keep track of the information required by the program that it represents, so that it can be executed. This is not actually the truth. Tasks are being executed, not programs. And they are many, not just one, though the number of CPUs (resources) is steady and limited and a single CPU can execute only one task at a time. So tasks compete with each other for the available resources. The question that may arise now is what happens to the tasks that did not capture a resource?

Of course they do not just disappear, but they wait in a CPU's **runqueue**, until it is their turn to be executed. These tasks are called Running tasks. The **current**[3] task is also a Running one. Running is one of the many task states. There are many

[1] Most commonly referred as CFS. It means Completely Fair Scheduler

[2] A task is a synonym to process. In the Linux kernel code the term task is used

[3] In the Linux kernel code, current, is the task that is currrently being executed

such states, but those that need to be mentioned in this thesis are the following:

- **Running**: tasks that are currently being executed, or waiting to be executed (in the kernel, it is represented by the `TASK_RUNNING` macro).

- **Interruptible**: it is one of the two sleeping states. It is waiting for a certain event to occur and then it will be awoken by an external signal (kernel representation: `TASK_INTERRUPTIBLE`).

- **Uninterruptible**: this is the second sleeping state. It is similar to the previous one and it is not commonly used. The difference between the two, is that such a task disables all external signals to it and it can only be awoken by the kernel [2]. These tasks, are usually waiting for a device, to finish its work. (kernel representation: `TASK_UNINTERRUPTIBLE`).

Other than state, tasks, also have a **priority** number, which ranges from 0 to 139[4]. Tasks with priority ranging from 0 to 99, are called **real time**, while the rest are called **normal**. Normal tasks, are then categorized as **batch** (they require much CPU time) or as **interactive** (these require interaction, e.g. user input). This distinction is not a strict one, as tasks with batch and interactive characteristics, at the same time, can appear. Also note here, that the lowest the priority value, the highest the task priority is.

Each of the two schedulers has a different way of calculating priorities, which I will discuss later. What they have in common is a field called `static_prio`, which is issued by the user [4]. Keep this in mind for now.

There are many more, one can say about tasks, though this is not this thesis' goal. I will direct you to the bibliography if you want to learn more.

## 2.2 Context Switch

Suppose that a task is now running. After $t$ time units, a new task arrives. The scheduler concludes, that the newly arrived task is better suited to run than the current task, so it sends a preemption signal to the current task. So now it is time, for the newly arrived task to run.

Before this can happen, an important step must be taken. The current must withdraw all of its data from the memory, so that the new one can occupy it with its own data. This is called **context switch** and it may take place after a task preemption.

One important thing about context switch, is that it needs a portion of time to finish. It is a rather small amount (around 0.01ms according to [8] and [9] [10] (search for `MIN_TIMESLICE`)) during which, though, the CPU is inactive. And taking into consideration, the amount of tasks running from a computer's boot until its

[4]This is how the kernel depicts it

termination and that tasks, actually run for some tens to hundreds of milliseconds, it can result in a huge impact on CPU usage.

## 2.3 Scheduler Invocation

There are two ways of invoking the main scheduler function: the direct and the indirect (or lazy) invocation.

In the first case, another function makes a direct call to the main scheduler function, while in the later case a function just sets the `TIF_NEED_RESCHED` flag and the main scheduler function, will be called in the near future.

## 2.4 The O(1) Scheduler

On January 4th 2002, the O(1) scheduler was announced as the replacement for the O(N) scheduler [3]. It was designed by **Ingo Molnar** and, as its name suggests, it is able to schedule tasks in a constant time. Instead of searching the entire runqueue, like its predecessors, for the next suitable task, it just performs some masking mechanisms. It was used until kernel version 2.6.23. I will discuss about the 2.6.0 version.

### 2.4.1 The O(1) runqueue

To better understand how the task type structure of the O(1) scheduler works, I must explain how the runqueue is implemented.

The runqueue, in O(1), is similar to a list (actually, it works as a list, it is not a list) and it keeps track of all tasks, in the `TASK_RUNNING` state. When a task enters one of the two sleep states, it exits the runqueue, and it will be reinserted when it becomes running again. Some of the fields included in the runqueue type structure are the following:

```
1 struct runqueue {
2     unsigned long nr_running;
3     unsigned long nr_switches;
4     task_t *curr;
5     prio_array_t *active, *expired;
6     ...
7 }
```

Listing 2.1: The O(1) runqueue

First I will explain how each of these fields, is used and later I will present their types:

- `nr_running`: a field that represents the number of tasks in `TASK_RUNNING` state.
- `nr_switches`: the number of context switches, that have taken place.

- `curr`: it is a pointer to the currently running task. It is of `task_struct` type (`task_t` is just an alias to it).

- `active`: basically, it is a list, that stores all tasks, which have not yet exceeded their `time_slice` (discussed in next section 2.4.2).

- `expired`: tasks that have exceeded their `time_slice`, end up being stored here.

Now you may have noticed the `prio_array_t` type structure. It is nothing more than an alias to the `struct prio_array` type structure, which implements the list fields of the runqueue. As its name suggests, tasks are queued in it according to their priority. It consists of the following fields:

```
1 struct prio_array {
2     int nr_active;
3     unsigned long bitmap[BITMAP_SIZE];
4     struct list_head queue[MAX_PRIO];
5 }
```

Listing 2.2: The priority array structure

- `nr_active`: this keeps track of the number of tasks stored in the `queue` field.

- `queue`: imagine it as a 2D array. It is an array of `MAX_PRIO` (or 140) pointers. Each of these pointers, stores tasks of the according priority. So at `queue[102]`, tasks of priority number 102 will be stored. For the O(1) scheduler, this priority refers to the dynamic priority of the task, which is discussed in the next section (2.4.2).

- `bitmap`: some pointers of the `queue` array may be empty. It would take `MAX_PRIO` times at worst case to search for the next non-empty pointer in the array, hence the scheduler would not be called O(1). So it uses a bitmapping mechanism. The `queue` has a size of 140, so the `bitmap` must consist of 140 bits at least, so as to track the next non-empty pointer. The index of the least significant of the set bits (equal to 1), indicates the index of the first non-empty pointer in the `queue` array.

Let's examine now the implementation of the list type structure in the kernel:

```
1 struct list_head {
2     struct list_head *next, *prev;
3 };
```

Listing 2.3: List in kernel code

So there is no pointer to the task type structure. How can the runqueue detect a task, if no pointer to its type structure exists? In the next section, you will see that a task is linked to the `active` or `expired` list fields, of the runqueue, via a list field included in the task type structure. I will discuss more about this later.

### 2.4.2 The O(1) task representation

Before I explain how the O(1) scheduler works, I will describe the basic fields of the most fundamental type structures used by the scheduler.

The first and most important is the `task_struct`. It is the kernel source code representation of a task. If you search in the source code for this structure, you will see that it is literally a huge structure, of around 250 lines, so this means, that it needs a lot of memory to store its data. In a way to save memory for the kernel, each declared variable of this type structure is a pointer, to a memory address, containing the task's data. Some of the most important fields are the following:

```
struct task_struct {
    unsigned long state;
    int prio, static_prio;
    unsigned list_head run_list;
    prio_array_t* array;
    unsigned long sleep_avg;
    unsigned long long timestamp;
    int activated;
    unsigned int time_slice;
    long interactive_credit;
    ...
}
```

Listing 2.4: O(1) task structure

So let me explain what each of these fields does:

- `state`: this field keeps track of the task's state.

- `prio`: or dynamic priority. It is one of the two priority fields in the O(1) scheduler. This is what the scheduler uses, in order to make any computations concerning task scheduling. It is calculated using the `effective_prio` function. For real time tasks it is the same as the static priority, while for normal tasks it is calculated using the following formula (where t is the task):

$$prio(t) = max(100,\ min(t.static\_prio - BONUS(t) + 5,\ 139)) \tag{2.1}$$

  So the dynamic priority is dependent on the task's static priority and a value called BONUS. The BONUS, is a value that may increase or decrease a task's priority, depending on the `sleep_avg` field. It is calculated as shown below:

$$BONUS(t) = t.sleep\_avg * 0.010 \tag{2.2}$$

  Its values range from 0 to 10 and it is basically, used to boost tasks, that have been asleep for too long.

- `run_list`: this field is used to link the task in the runqueue (to be more precise, it is used to link the task with the `active` or `expired` fields of the runqueue). The `prev` pointer (remember the list type structure), points to the previous task's `run_list` and the `next` points to the next one.

- `array`: when the task has exhausted its time slice, it points to the runqueue's `expired` field. If it has not exhausted its time slice and it is waiting to be executed, it points to the runqueue's `active` field. In both cases it is considered to be a running task.

- `sleep_avg`: basically it is not the actual average time that the task has been asleep, but the total time the task slept. Its value, though, may change due to some bonuses O(1) gives to a task, according to its sleep time. For example, when a task is executed again, at some point, its run time is deducted from its `sleep_avg` value.

- `timestamp`: this has two roles. It may represent the last time the task was context switched, or the last time the task entered the runqueue. This will become more clear later.

- `interactive_credit`: this is a bonus given to tasks that are considered interactive. Its value is depended indirectly on the task's sleep time (`sleep_avg` field). This bonus decreases after some time.

- `activated`: it is a certain code that the task receives when it awakes depending on, whether it was interruptible and woke up by system call (1 as value) or uninterruptible (-1 as value) or it was interruptible and it is being woken up by an interrupt. The default value is 0. It can be used to give sleep bonuses to the task.

- `time_slice`: this is the total amount of ticks that a task can run, before it can become expired. A task's time slice is dependent on the task's static priority. It is calculated using the following formula (from [1]):

$$time\_slice(t) = \begin{cases} (140 - t.static\_prio) * 20, & \text{if } t.static\_prio < 120 \\ (140 - t.static\_prio) * 5, & \text{if } t.static\_prio \geq 120 \end{cases} \tag{2.3}$$

  There are many more fields contained in the `task_struct`, as stated before, which is not part of this thesis goal to discuss.

### 2.4.3 How O(1) works

Now that I have explained how the basic scheduler type structures are implemented, it is time to start discussing about, how the O(1) works. Some of the most fundamental functionalities of this scheduler are the scheduler tick, dynamic priority recalculation, wake up and schedule mechanisms.

#### Scheduler Tick

The scheduler **tick** is implemented by the `scheduler_tick` function. It is called by the timer with `HZ` frequency [10], where `HZ` is a predefined value in the kernel code. Its primary job is to check if the current task has exceeded its time slice and

if that is the case, then make an indirect invocation to the scheduler, by setting the `TIF_NEED_RESCHED` macro, to 1. The following are a part of the function:

```
void scheduler_tick(int user_ticks, int sys_ticks)
{
    ...
    task_t *p = current;
    ...
    if (!--p->time_slice) {
    dequeue_task(p, rq->active);
    set_tsk_need_resched(p);
    p->prio = effective_prio(p);
    p->time_slice = task_timeslice(p);
    p->first_time_slice = 0;
        ...
    if (!TASK_INTERACTIVE(p) || EXPIRED_STARVING(rq)) {
        enqueue_task(p, rq->expired);
    } else
        enqueue_task(p, rq->active);
    }
    // Continued in the 2nd part
```

Listing 2.5: Scheduler tick function part 1

At this part, the tick function, checks if the current task has depleted its time slice, after reducing it by one. As I said at 2.4.2 section, the `time_slice` of each task, is measured as tick amount. If it is depleted then, it dequeues the task from the runqueue, sets the `TIF_NEED_RESCHED` macro and recalculates the task's time slice (using the `task_timeslice` function) and dynamic priority (using the `effective_prio` function) using the 2.3 and 2.1 formulas accordingly. Then it checks if the current task is interactive. If it is, it is reinserted at `active` priority array of the runqueue. On the other hand if it is not, it is queued on the `expired` priority array.

```
    else {
   /*
     * Prevent a too long timeslice allowing a task to monopolize
     * the CPU. We do this by splitting up the timeslice into
     * smaller pieces.
     *
     * Note: this does not mean the task's timeslices expire or
     * get lost in any way, they just might be preempted by
     * another task of equal priority. (one with higher
     * priority would have preempted this task already.) We
     * requeue this task to the end of the list on this priority
     * level, which is in essence a round-robin of tasks with
     * equal priority.
     *
     * This only applies to tasks in the interactive
     * delta range with at least TIMESLICE_GRANULARITY to requeue.
     */
       if (TASK_INTERACTIVE(p) && !((task_timeslice(p) -
            p->time_slice) % TIMESLICE_GRANULARITY(p)) &&
            (p->time_slice >= TIMESLICE_GRANULARITY(p)) &&
            (p->array == rq->active)) {

            dequeue_task(p, rq->active);
            set_tsk_need_resched(p);
```

```
25              p->prio = effective_prio(p);
26              enqueue_task(p, rq->active);
27          }
28      }
29  }
```

Listing 2.6: Scheduler tick function, part 2

The execution, flows from this part, only when the task, has not depleted its time slice. It checks if the current task is interactive and if its time slice meets certain requirements, which check, as mentioned in the comments, can be described as a measurement of the ticks that the current task has to be requeued in one of the priority arrays of the runqueue. If the condition is valid, then it recalculates the current task's dynamic priority and requeues it on the `active` priority array.

I need to note here, that the enqueue operation, adds a task at the tail (or end) of the according priority list.

### Dynamic priority recalculation

It is performed by the `recalc_task_prio` function. This funtion is called every time the main scheduler function is invoked, or when a task wakes up. It is some kind of wrapper around the `effective_prio` function, that also calculates some other statistics.

```
1  static void recalc_task_prio(task_t *p, unsigned long long now)
2  {
3      unsigned long long __sleep_time = now - p->timestamp;
4      unsigned long sleep_time;
5
6      if (__sleep_time > NS_MAX_SLEEP_AVG)
7          sleep_time = NS_MAX_SLEEP_AVG;
8      else
9          sleep_time = (unsigned long)__sleep_time;
```

Listing 2.7: Dynamic priority recalculation part 1

Firstly, it calculates the sleep time of the task (not the current one). There is also an upper limit to it, called `NS_MAX_SLEEP_AVG`. The value of this macro is equal to `MAX_SLEEP_AVG`, if converted to nanoseconds. The secondly mentioned macro, is equivalent to `AVG_TIMESLICE * MAX_BONUS`, which corresponds to $102 * 10$, if all values are converted from jiffies to milliseconds (just divide jiffies values with the `HZ` constant value, to convert them to milliseconds).

After that, it checks if the calculated `sleep_time` is bigger than 0 and enters the condition's block. Otherwise it just calls `effective_prio` and exits. Note that this function is called even if the condition is valid. Let's examine what is contained in this `if` block:

```
1      if (p->mm && p->activated != -1 &&
2          sleep_time > JUST_INTERACTIVE_SLEEP(p)){
3              p->sleep_avg = JIFFIES_TO_NS(MAX_SLEEP_AVG -
4              AVG_TIMESLICE);
```

```
5             if (!HIGH_CREDIT(p))
6                 p->interactive_credit++;
7     // An else statement follows next
```

Listing 2.8: Dynamic priority recalculation part 2

So it jumps right onto another `if` statement, then followed by another one. The first one, checks whether the task is not a kernel thread (with `p->mm`), then if it was in `TASK_UNINTERRUPTIBLE` state, before it was woken up (`p->activated` with code -1 means, that the task was in the uninterruptible state before it was woken up, 0 is the default and 1 means that it woke by an interrupt) and finally, whether its sleep time has exceeded an upper sleep time limit, after which a task is considered as interactive. Then, the second statement, checks whether the woken up interactive task needs to get some bonuses, in order to save from its sleep time.

We can understand from all that, that the O(1) scheduler, detects interactive tasks by measuring their sleep time. Each task with a different static priority, has a different sleep time limit, after which it can be considered as interactive. This limit is calculated by the `JUST_INTERACTIVE_SLEEP` macro:

```
1 // I simplified the DELTA macro a bit:
2 #define DELTA(t) (NICE(t->ts) / (PRIO_BONUS_RATIO / 100.0)) + 2
3
4 #define JUST_INTERACTIVE_SLEEP(p) \
5     (JIFFIES_TO_NS(MAX_SLEEP_AVG * \
6         (MAX_BONUS / 2 + DELTA((p)) + 1) / MAX_BONUS - 1))
```

Listing 2.9: Interactive sleep limit macro

Before I continue explaining how the function works, it would be useful to demonstrate how this credit, or bonus, for the so called interactive tasks works. `interactive_credit` is manipulated with two macros called `HIGH_CREDIT` and `LOW_CREDIT`. They are used to define whether the task needs more credit on certain fields, like the `sleep_avg` field. When talking about credit, I mean an increase or a decrease to the field's value, depending on what the increment or the decrement will mean to the value when depicted as a bonus. For the `sleep_avg` field, for example, an increase to its value, will mean higher dynamic priority. You can see how the two macros work below:

```
1 #define CREDIT_LIMIT    100
2
3 #define HIGH_CREDIT(p) \
4     ((p)->interactive_credit > CREDIT_LIMIT)
5
6 #define LOW_CREDIT(p) \
7     ((p)->interactive_credit < -CREDIT_LIMIT)
```

Listing 2.10: Credit manipulation macros

Now that I explained better how `interactive_credit` works, it will be easier to understand the rest of the function. So back to the dynamic priority recalculation mechanism:

```
1     } else {
2         /*
3          * The lower the sleep avg a task has the more
4          * rapidly it will rise with sleep time.
5          */
6         sleep_time *= (MAX_BONUS - CURRENT_BONUS(p)) ? : 1;
7
8         /*
9          * Tasks with low interactive_credit are limited to
10         * one timeslice worth of sleep avg bonus.
11         */
12        if (LOW_CREDIT(p) &&
13            sleep_time > JIFFIES_TO_NS(task_timeslice(p)))
14                sleep_time =
15                    JIFFIES_TO_NS(task_timeslice(p));
16
17        /*
18         * Non high_credit tasks waking from uninterruptible
19         * sleep are limited in their sleep_avg rise as they
20         * are likely to be cpu hogs waiting on I/O
21         */
22        if (p->activated == -1 && !HIGH_CREDIT(p) && p->mm){
23            if (p->sleep_avg >= JUST_INTERACTIVE_SLEEP(p))
24                sleep_time = 0;
25            else if (p->sleep_avg + sleep_time >=
26                JUST_INTERACTIVE_SLEEP(p)){
27                    p->sleep_avg =
28                        JUST_INTERACTIVE_SLEEP(p);
29                    sleep_time = 0;
30                }
31        }
32
33        /*
34         * This code gives a bonus to interactive tasks.
35         *
36         * The boost works by updating the 'average sleep time'
37         * value here, based on ->timestamp. The more time a task
38         * spends sleeping, the higher the average gets - and the
39         * higher the priority boost gets as well.
40         */
41        p->sleep_avg += sleep_time;
42
43        if (p->sleep_avg > NS_MAX_SLEEP_AVG){
44            p->sleep_avg = NS_MAX_SLEEP_AVG;
45            if (!HIGH_CREDIT(p))
46                p->interactive_credit++;
47        }
48    }
```

Listing 2.11: Dynamic priority recalculation part 3

Whichever of the first two conditions is valid, all lead to the incrementation of the task's `sleep_avg` by `sleep_time`.

This literally was the most well documented part of the kernel code that I found. The one thing that might seem to misbehave is line 6. When the condition in the ternary operator is valid, it returns the value `MAX_BONUS - CURRENT_BONUS(p)`,

else it returns 1. The `CURRENT_BONUS` macro is using the formula 2.2, that I already mentioned. And if you remember, the bonus each task receives is dependent on its `sleep_avg`. The higher it is the higher the bonus and the higher the bonus of a task the slower is its `sleep_avg` being increased, so as to slow down the bonus incrementation process and thus not allowing the task to monopolize the CPU.

You can also see here the `LOW_CREDIT` macro in use. If the task has a low `interactive_credit` value, then it limits its `sleep_avg`, so as not to get a higher bonus value too soon.

After all of these calculations, it just calls the `effective_prio` function, to recalculate the task's priority and enqueue it to the corresponding priority array.

### Wake up

This is a function, that may, or may not, change the state of a task to `TASK_RUNNING`. The main wake up function in the kernel code is called `try_to_wake_up`. It is always called "indirectly", using other wake up wrapper functions such as the `wake_up_process` or the `wake_up_state` wrappers. Its job is quite simple. It just inserts the woken up task in the runqueue, checks if it has more rights to run than the current task, sets its state to running and returns a success code, depending on the outcome of the wake up operation (the task might not have woken up).

```
1 static int try_to_wake_up(task_t * p, unsigned int state, int sync)
2 {
3     int success = 0;
4     long old_state;
5     // there are two more variables
6     ...
7     old_state = p->state;
```

Listing 2.12: The main wake up function part 1

As parameters, it receives the task to be woken up and a "list" of the task states the interrupt is associated with (marked by the `state` parameter). By "list", here I mean all the states that are associated with the interrupt, combined together via a bitwise and operation. For example, if the interrupt is associated with `TASK_INTERRUPTIBLE` and `TASK_UNINTERRUPTIBLE` states, then the function will be called like this:

```
try_to_wake_up(p, TASK_INTERRUPTIBLE & TASK_UNINTERRUPTIBLE, sync)
```

The third line, of the below code, concerns a synchronized wake up, which does not interest us right now.

```
1     if (old_state & state) {
2         if (!p->array) {
3             ...
4             if (old_state == TASK_UNINTERRUPTIBLE){
5                 rq->nr_uninterruptible--;
6                 /*
7                  * Tasks on involuntary sleep don't earn
8                  * sleep_avg beyond just interactive state.
```

```
                 */
                p->activated = -1;
            }
            // Another if follows here, accompanied by the
            // following else statement
            else {
                activate_task(p, rq);
                if (TASK_PREEMPTS_CURR(p, rq))
                    resched_task(rq->curr);
            }
            success = 1;
        }
        p->state = TASK_RUNNING;
    }

    return success;
}
```

Listing 2.13: The main wake up function part 2

Do not let the condition `!p->array` trouble you. Recall that a task is being dequeued from the runqueue once it enters one of the sleep states, so its `array` pointer shows to no memory address. In other words, this pointer, is a way of discriminating running from sleeping tasks.

What was left unexplained about the above code are the `TASK_PREEMPTS_CURR` macro, `resched_task` and `activate_task` functions. The first one just compares the dynamic priorities of the current task and the task that is being awakened, to check which one has the highest priority, to run next (to be the new current task). The second, can be described as a long way[5] of setting the `TIF_NEED_RESCHED` to 1. The third is a bit more complicated:

```
static inline void activate_task(task_t *p, runqueue_t *rq)
{
    unsigned long long now = sched_clock();

    recalc_task_prio(p, now);

    /*
     * This checks to make sure it's not an uninterruptible task
     * that is now waking up.
     */
    if (!p->activated){
        /*
         * Tasks which were woken up by interrupts (ie. hw events)
         * are most likely of interactive nature. So we give them
         * the credit of extending their sleep time to the period
         * of time they spend on the runqueue, waiting for
   execution
         * on a CPU, first time around:
         */
        if (in_interrupt())
            p->activated = 2;
```

[5]Only if the multiprocessing configurations are enabled, which I will not cover, so in our case it just sets the `TIF_NEED_RESCHED`

```
21          else
22          /*
23           * Normal first-time wakeups get a credit too for on-
    runqueue
24           * time, but it will be weighted down:
25           */
26              p->activated = 1;
27      }
28      p->timestamp = now;
29
30      __activate_task(p, rq);
31  }
```

Listing 2.14: Task activation

It first calculates the dynamic priority of the newly activated task, then checks if the task was previously sleeping and updates its `activated` field, according to the way the task was awakened. The `timestamp` field is also updated and takes the meaning of "time the task is inserted in the runqueue"[6]. Finally, it calls the `__activate_task` function, that just inserts the task at the tail of the corresponding priority array.

### Schedule

Last but not least and most important, the `schedule` function. It has the role, of choosing the next current task and then context switch the previous current with the next one. It is the main scheduler function [10] and it is the one called either directly, or indirectly (lazy invocation). It is divided into three parts (a more logical division, based on tags): the `need_resched`, `switch_tasks` and `pick_next_task`.

```
1  asmlinkage void schedule(void)
2  {
3      task_t *prev, *next;
4      prio_array_t *array;
5      unsigned long long now;
6      unsigned long run_time;
7
8  need_resched:
9      preempt_disable();
10     prev = current;
11
12     ...
13
14     now = sched_clock();
15     if (likely(now - prev->timestamp < NS_MAX_SLEEP_AVG))
16         run_time = now - prev->timestamp;
17     else
18         run_time = NS_MAX_SLEEP_AVG;
19
20     /*
21      * Tasks with interactive credits get charged less run_time
22      * at high sleep_avg to delay them losing their interactive
23      * status
24      */
```

[6]The function `sched_clock` is responsible, for returning the CPU time

```
25      if (HIGH_CREDIT(prev))
26          run_time /= (CURRENT_BONUS(prev) ? : 1);
27
28      ...
29
30      /*
31       * if entering off of a kernel preemption go straight
32       * to picking the next task.
33       */
34      if (unlikely(preempt_count() & PREEMPT_ACTIVE))
35          goto pick_next_task;
36
37      switch (prev->state) {
38      case TASK_INTERRUPTIBLE:
39          if (unlikely(signal_pending(prev))) {
40              prev->state = TASK_RUNNING;
41              break;
42          }
43      default:
44          deactivate_task(prev, rq);
45          prev->nvcsw++;
46          break;
47      case TASK_RUNNING:
48          prev->nivcsw++;
49      }
```

Listing 2.15: The schedule function part 1

In the `need_resched` section, all that the scheduler does is to check, if the current task can be preempted by another. It also calculates some statistics. First, it disables all interrupts and calculates the run time of the current task. Remember, that `timestamp`, is, at this point, the time when the current task, started its execution. Note, that tasks, viewed as interactive by the scheduler, are charged with lower run time. Then, according to the task's state, it just updates the involuntary and voluntary context switch, statistics and continues to the next section.

```
1  pick_next_task:
2      if (unlikely(!rq->nr_running)) {
3          ...
4          next = rq->idle;
5          rq->expired_timestamp = 0;
6          goto switch_tasks;
7      }
8
9      array = rq->active;
10     if (unlikely(!array->nr_active)) {
11         /*
12          * Switch the active and expired arrays.
13          */
14         rq->active = rq->expired;
15         rq->expired = array;
16         array = rq->active;
17         rq->expired_timestamp = 0;
18     }
19
20     idx = sched_find_first_bit(array->bitmap);
21     queue = array->queue + idx;
```

```
22      next = list_entry(queue->next, task_t, run_list);
23
24      if (next->activated > 0) {
25          unsigned long long delta = now - next->timestamp;
26
27          if (next->activated == 1)
28              delta = delta * (ON_RUNQUEUE_WEIGHT * 128 / 100) / 128;
29
30          array = next->array;
31          dequeue_task(next, array);
32          recalc_task_prio(next, next->timestamp + delta);
33          enqueue_task(next, array);
34      }
35      next->activated = 0;
```

Listing 2.16: The schedule function part 2

The second section, attempts to find the most suitable to run next task. If no task is left in the runqueue, then the CPU remains idle (the next task will be the idle task, which you can imagine as a dummy task, that is used in order not to cause problems in the next context switch). The next statement checks whether the priority array of the active tasks is not empty. If it is, then it switches it with the priority array of the expired tasks. Finally, it searches for the first non empty priority queue and picks the the first task (FIFO) and also gives it some bonuses, if it was previously interrupted and recalculates its priority (the `timestamp` for the next task is the time when it was inserted in the runqueue).

```
1  switch_tasks:
2      ...
3      clear_tsk_need_resched(prev);
4      ...
5
6      prev->sleep_avg -= run_time;
7      if ((long)prev->sleep_avg <= 0){
8          prev->sleep_avg = 0;
9          if (!(HIGH_CREDIT(prev) || LOW_CREDIT(prev)))
10             prev->interactive_credit--;
11     }
12     prev->timestamp = now;
13
14     if (likely(prev != next)) {
15         next->timestamp = now;
16         rq->nr_switches++;
17         rq->curr = next;
18
19         prepare_arch_switch(rq, next);
20         prev = context_switch(rq, prev, next);
21         barrier();
22
23         finish_task_switch(prev);
24     } else
25         ...
26
27     ...
28     if (test_thread_flag(TIF_NEED_RESCHED))
29         goto need_resched;
```

```
}
```

Listing 2.17: The schedule function part 3

The final section performs the context switch and later checks if the `TIF_NEED_RESCHED` flag was set again, before it exits.

## 2.5 The CF Scheduler

This is the currently used scheduler in the Linux kernel. It was introduced in the 2.6.23 kernel version and was designed, again, by Ingo Molnar in 2007. It has a O(log(n)) time complexity. It was based on the idea that "it is physically impossible to run any more than one execution flow on a single processor (core). So, CFS tries to mimic perfectly fair scheduling. Rather than simply assign a time slice to a process, the scheduler calculates how long a task should run as a function of the total number of currently runnable processes." [4]. I will discuss about the 2.6.24 version.

### 2.5.1 The CFS runqueue

In contrast to the rest of the schedulers, CFS implements the runqueue, not as a list, but as a red black tree. This is the reason for its logarithmic time complexity. Of course the use of the runqueue is still the same, but many things differ, when comparing it to the runqueue implemented in the O(1) scheduler.

Part of the C type structure, of the CFS runqueue, in the kernel is the following:

```
struct cfs_rq {
    struct load_weight load;
    unsigned long nr_running;

    u64 exec_clock;
    u64 min_vruntime;

    struct rb_root tasks_timeline;
    struct rb_node *rb_leftmost;
    /* 'curr' points to currently running entity on this cfs_rq.
     * It is set to NULL otherwise (i.e when none are currently
   running).
     */
    struct sched_entity *curr;
```

Listing 2.18: The CFS runqueue

Now, let me explain what each of them is used for:

- `load`: every task has its own load. This field is the total sum of these loads. I will discuss more about load in the next section.

- `nr_running`: this is the total number of tasks in the runqueue.

- `exec_clock`: this is mostly used for statistics. It keeps track of the amount of time the runqueue's tasks were being executed.

- `min_vruntime`: this field will be fully understood later. Just keep for now, that it is not the minimum virtual run time of all tasks and that it is designed to increase monotonically.

- `tasks_timeline`: this is the root of the red black tree, containing the running tasks.

- `rb_leftmost`: the leftmost node of the red black tree and also the node with the task to run next.

- `curr`: the current task. It is of `sched_entity` type, which is very similar to a `task_struct`. The difference is that an entity may include more than one tasks, while a `task_struct` can describe only one task. To keep things simple, we will view it as a `task_stuct` and it will be described in the next section.

And how is the red black tree type structure implemented in the kernel?

```
struct rb_node
{
    unsigned long  rb_parent_color;
#define RB_RED    0
#define RB_BLACK  1
    struct rb_node *rb_right;
    struct rb_node *rb_left;
} __attribute__((aligned(sizeof(long))));
    /* The alignment might seem pointless, but allegedly CRIS needs
    it */

struct rb_root
{
    struct rb_node *rb_node;
};
```

Listing 2.19: Red black tree implementation

The implementation details do not interest us, but what might have got your attention is that, again, there is no pointer to a `sched_entity` type structure, or to a task. The task itself keeps track of its position in the tree and a masking mechanism is used to extract the whole `sched_entity` type structure, from the red black tree pointer.

### 2.5.2 The CFS task representation

When CFS was designed, some more type structures were introduced. These new type structures, were created, in order to make the implementation of new schedulers an easier job and also to boost the scheduler's performance. For example, the `sched_entity` structure, which we are going to discuss, was introduced, as a means for group scheduling, in CFS and it is included as a field in the `task_struct` type

structure.

CFS interacts with tasks, using their `sched_entity` field, which contains all important statistics, for CF scheduling. It is a simple type structure and far smaller, than `task_struct`.

```
struct sched_entity {
    struct load_weight  load;   /* for load-balancing */
    struct rb_node    run_node;
    unsigned int    on_rq;

    u64     exec_start;
    u64     sum_exec_runtime;
    u64     vruntime;
    u64     prev_sum_exec_runtime;
    ...
```

Listing 2.20: The CFS scheduling entities

The above are some of the most notable fields. There are many more, used only for statistical analysis and some for enabling group scheduling, which I am not going to discuss.

- `load`: it is used as a measure of the importance of a task. It is a constant value assigned to a task, according to its priority (its static priority. There is no dynamic priority in CF scheduling). It is implemented like this:

```
struct load_weight {
    unsigned long weight, inv_weight;
};
```

Listing 2.21: The CFS load weight

  where the `weight` field, holds the aforementioned value. These values are contained in an array, called `prio_to_weight`. Each of its indexes, is accessed using a task's priority.

- `run_node`: the position of the task's node in the red black tree.

- `on_rq`: this determines, whether the task is loaded on one of the runqueues or not. Note here, that the current task is considered to be on the runqueue. Save this note for later, as it will be needed and be better understood then.

- `exec_start`: the timestamp, when the task started being executed, since the last tick.

- `sum_exec_runtime`: this is the total execution time of a task.

- `prev_sum_exec_runtime`: like the previous field, but it refers to its previous value.

- `vruntime`: or virtual run time. This field is used for task scheduling by CFS.

It is a virtual measurement of time and together with the `min_vruntime` field of the runqueue, it implements the virtual clock in an abstract way. The `update_curr` function is used to update this value.

With the conclusion of this section, some things may still be unclear, which I will try to explain in the next section.

### 2.5.3 How CFS works

Before I start explaining, how the basic functions work, I need to first explain two fundamental terms, that constitute CFS: the virtual clock and latency tracking.

#### Virtual Clock

The virtual clock "measures the amount of time a waiting process would have been allowed to spend on the CPU on a completely fair system"[2]. As I mentioned earlier, it is completely abstract (there is no such type structure) and instead various fields of different type structures are used to implement it.

The function used for virtual time calculations, is called `update_curr`. It is used to recalculate the `vruntime` and `min_vruntime` fields and it is called at every tick. Also note, that these values, increase with every tick and that only the current task's fields are updated.

Tasks are inserted in the red black tree with their key equal to $vruntime - min\_vruntime$. The leftmost node is the one with the minimum key and the one to run next. When combining the previous note (note, from the `on_rq` task field), with all that have been mentioned in this section, we can assume that the current task, is slowly moving to the right of the red black tree. And this is the basic idea of CF scheduling.

Now we will take a closer look at the `update_curr` function and how it calculates the virtual run times.

```
1  static void update_curr(struct cfs_rq *cfs_rq)
2  {
3      struct sched_entity *curr = cfs_rq->curr;
4      u64 now = rq_of(cfs_rq)->clock;
5      unsigned long delta_exec;
6
7      if (unlikely(!curr))
8          return;
9
10     /*
11      * Get the amount of time the current task was running
12      * since the last time we changed load (this cannot
13      * overflow on 32 bits):
14      */
15     delta_exec = (unsigned long)(now - curr->exec_start);
16
17     __update_curr(cfs_rq, curr, delta_exec);
18     curr->exec_start = now;
```

```
19
20      ...
21  }
22
23  static inline void
24  __update_curr(struct cfs_rq *cfs_rq, struct sched_entity *curr,
25            unsigned long delta_exec)
26  {
27      unsigned long delta_exec_weighted;
28      u64 vruntime;
29
30      schedstat_set(curr->exec_max, max((u64)delta_exec, curr->
        exec_max));
31
32      curr->sum_exec_runtime += delta_exec;
33      schedstat_add(cfs_rq, exec_clock, delta_exec);
34      delta_exec_weighted = delta_exec;
35      if (unlikely(curr->load.weight != NICE_0_LOAD)) {
36          delta_exec_weighted = calc_delta_fair(delta_exec_weighted,
37                                &curr->load);
38      }
39      curr->vruntime += delta_exec_weighted;
40
41      /*
42       * maintain cfs_rq->min_vruntime to be a monotonic increasing
43       * value tracking the leftmost vruntime in the tree.
44       */
45      if (first_fair(cfs_rq)) {
46          vruntime = min_vruntime(curr->vruntime,
47                  __pick_next_entity(cfs_rq)->vruntime);
48      } else
49          vruntime = curr->vruntime;
50
51      cfs_rq->min_vruntime =
52          max_vruntime(cfs_rq->min_vruntime, vruntime);
53  }
```

Listing 2.22: The virtual run time update function

It may seem a lot, but it is a simple function. Firstly it calculates the execution delta of the current task, since the last tick (note that it later changes the `exec_start` field to now) and then calls the `__update_curr` function, which is responsible for the virtual run time calculations. Some parts of this function concern the calculation of some scheduling statistics, which I will bypass, as they are not that important in the scheduling process. The function starts by updating the runqueue's `exec_clock` (adds the `delta_exec` value to it) and by weighting the `delta_exec` value. If the task has a priority number of 120 (0 in nice), then the weighted value (the task's weight is equal to `NICE_0_LOAD` 1024) is the same as `delta_exec`, else it uses the following formula for the weighting:

$$delta\_exec\_weighted = delta\_exec * \frac{1024}{curr \rightarrow load.weight}$$

and then the weighted execution delta is added to the task's virtual run time field and thus it is gradually moving to the right parts of the tree.

Then it needs to calculate the `min_vruntime`, which, as I mentioned, increases monotonically. The `first_fair` function is used to check whether a leftmost task exists or not. This is because the current task, is removed from the red black tree (but it is still considered to be inside it) and is reinserted in it when it stops executing. So this function is used, to avoid some calculations, regarding choosing the minimum `vruntime` between the leftmost and the current task. If there is no other task left in the runqueue, then the minimum value is known and is equal to the current task's virtual run time. After that, the previously found minimum `vruntime` is compared to the runqueue's current `min_vruntime` and the maximum value is chosen to be the next `min_vruntime`.

### Latency tracking

Latency is the time within which every task in the runqueue is promised to run at least one time. By default this value is equal to 20 milliseconds and the variable is named as `sysctl_sched_latency`. There are two more predefined variables. The first one is called `sched_nr_latency` which measures the amount of tasks, that can be handled at each latency and it is equal to 5 by default. The latency, can and will change each time the number of active (there are no expired tasks in CFS) tasks (`nr_running`), is larger than `sched_nr_latency`. The last of these variables is called `sysctl_sched_min_granularity` and its value is equal to $\frac{sysctl\_sched\_latency}{sched\_nr\_latency}$ and by default set to 4 milliseconds.

As mentioned, when there are too many tasks we have to stretch[11] latency, or the "time slices" will get very small. The recalculation is done with the help of the `__sched_period` function, whose work, can be described with the following formula:

$$latency = sysctl\_sched\_latency * \frac{nr\_running}{sched\_nr\_latency}$$

The function is called each time a new task is inserted in the runqueue.

Now, it is time to discuss about the basic operations of CFS.

### Scheduler tick

The CFS tick function is named `task_tick_fair`, which actually just passes the control flow to the `entity_tick` function.

```
1  static void entity_tick(struct cfs_rq *cfs_rq, struct sched_entity
       *curr)
2  {
3      /*
4       * Update run-time statistics of the 'current'.
5       */
6      update_curr(cfs_rq);
7
8      if (cfs_rq->nr_running > 1 || !sched_feat(WAKEUP_PREEMPT))
9          check_preempt_tick(cfs_rq, curr);
10 }
```

Listing 2.23: The entity tick function

It is quite simple. It calls the `update_curr` function and then checks whether there are more than one tasks in the runqueue (remember, that when there is only one running task, this task is the current task and that it is not on the runqueue) and if there are, calls the `check_preempt_tick` function.

```
static void
check_preempt_tick(struct cfs_rq *cfs_rq, struct sched_entity *curr
    )
{
    unsigned long ideal_runtime, delta_exec;

    ideal_runtime = sched_slice(cfs_rq, curr);
    delta_exec = curr->sum_exec_runtime -
        curr->prev_sum_exec_runtime;
    if (delta_exec > ideal_runtime)
        resched_task(rq_of(cfs_rq)->curr);
}
```

Listing 2.24: The check preempt tick function

The function just checks if the current task is no longer the one suitable to be running and if it is not, sets the `TIF_NEED_RESCHED` flag, to 1. The `sched_slice` function, calculates the ideal run time of the current task (it is the time that the task would run in a completely parallel system) using this formula:

$$latency = latency * \frac{curr \rightarrow load.weight}{cfs\_rq \rightarrow load.weight}$$

where latency is the one calculated by the `__sched_period` function. It returns the task's share of the latency period, that was calculated. So you may call its return value as the task's granularity.

### Scheduling (main scheduling function)

The main scheduling function's logic is similar to the one developed for the O(1) scheduler. The main difference is in the mechanism used, for picking the next task.

```
asmlinkage void __sched schedule(void)
{
    struct task_struct *prev, *next;
    long *switch_count;
    ...

    prev = rq->curr;
    switch_count = &prev->nivcsw;
    ...
    clear_tsk_need_resched(prev);
    if (prev->state && !(preempt_count() & PREEMPT_ACTIVE)) {
        if (unlikely((prev->state & TASK_INTERRUPTIBLE) &&
                unlikely(signal_pending(prev)))) {
            prev->state = TASK_RUNNING;
        } else {
            deactivate_task(rq, prev, 1);
        }
        switch_count = &prev->nvcsw;
    }
```

```
    ...
    prev->sched_class->put_prev_task(rq, prev);
    next = pick_next_task(rq, prev);
    ...
    if (likely(prev != next)) {
        rq->nr_switches++;
        rq->curr = next;
        ++*switch_count;

        context_switch(rq, prev, next); /* unlocks the rq */
    } else
        ...
    ...
    preempt_enable_no_resched();
    if (unlikely(test_thread_flag(TIF_NEED_RESCHED)))
        goto need_resched;
```

Listing 2.25: The main scheduler function

Its concept, as you may have noticed, is still the same. The first `if` block statement, resembles the `switch-case` block in the O(1). After these checks, the current task is put back into the runqueue, with the `put_prev_task` function and then the next current task is selected with the `pick_next_task` function. This is the only part of the `schedule` function that has drastically changed in logic. You may also have noticed the `sched_class` field of the `task_struct` type structure. This is a type structure, that contains pointers, to all the basic functions, that a scheduler (like CFS or the real time scheduler) needs to perform its scheduling policies. It was designed in order to make the process of developing a new scheduler easier.

```
struct sched_class {
    const struct sched_class *next;

    void (*enqueue_task) (struct rq *rq, struct task_struct *p, int
    wakeup);
    void (*dequeue_task) (struct rq *rq, struct task_struct *p, int
    sleep);
    void (*yield_task) (struct rq *rq);

    void (*check_preempt_curr) (struct rq *rq, struct task_struct *
   p);

    struct task_struct * (*pick_next_task) (struct rq *rq);
    void (*put_prev_task) (struct rq *rq, struct task_struct *p);

#ifdef CONFIG_SMP
    unsigned long (*load_balance) (struct rq *this_rq,
            int this_cpu,
            struct rq *busiest, unsigned long max_load_move,
            struct sched_domain *sd, enum cpu_idle_type idle,
            int *all_pinned, int *this_best_prio);

    int (*move_one_task) (struct rq *this_rq, int this_cpu,
                  struct rq *busiest, struct sched_domain *sd,
                  enum cpu_idle_type idle);
#endif

    void (*set_curr_task) (struct rq *rq);
```

```
26      void (*task_tick) (struct rq *rq, struct task_struct *p);
27      void (*task_new) (struct rq *rq, struct task_struct *p);
28  };
29
30  struct task_struct {
31      ...
32      const struct sched_class *sched_class;
33      ...
34  }
```

Listing 2.26: The scheduling class structure

These are all the functions needed for a scheduler to perform its policy. There is also a pointer to another `sched_class` structure, the next in priority scheduler. All schedulers in this way are linked together in a circular list. Each task, according to its priority, is assigned to a different scheduler. So for example normal tasks are assigned to CFS and real time tasks to the real time scheduler. This assignment is done by setting the `sched_class` field of the task's type structure to the according address.

The rest remain the same. So, there is no need to discuss anything further about this function.

According to the above, the `pick_next_task` of CFS's `sched_class` structure, will point to the function `pick_next_task_fair` (contained in kernel/sched_fair.c [11] file). This just actually calls the `pick_next_entity` function, which is shown below.

```
1  static struct sched_entity *pick_next_entity(struct cfs_rq *cfs_rq)
2  {
3      struct sched_entity *se = NULL;
4
5      if (first_fair(cfs_rq)) {
6          se = __pick_next_entity(cfs_rq);
7          set_next_entity(cfs_rq, se);
8      }
9
10     return se;
11 }
```

Listing 2.27: Function for picking the next current task

It checks, whether a leftmost node (or task) exists in the runqueue and calls some more functions. The `__pick_next_entity`, just returns the leftmost task as a `sched_entity` type. The other one is shown below:

```
1  static void
2  set_next_entity(struct cfs_rq *cfs_rq, struct sched_entity *se)
3  {
4      /* 'current' is not kept within the tree. */
5      if (se->on_rq) {
6          ...
7          __dequeue_entity(cfs_rq, se);
8      }
9
```

```
    ...
    cfs_rq->curr = se;
}
```

Listing 2.28: Set the next task

I bypassed some statistic calculations. The function, dequeues the task from the runqueue, as I already mentioned and sets the current task to the newly selected to run next task. Of course, this task, when it will no longer be suitable for running, must be inserted back into the runqueue. This is done with the `put_prev_task` function, which calls the `put_prev_task_fair` when CFS is used. The flow is passed to the `put_prev_entity` function, which is shown below:

```
static void put_prev_entity(struct cfs_rq *cfs_rq, struct
    sched_entity *prev)
{
    /*
     * If still on the runqueue then deactivate_task()
     * was not called and update_curr() has to be done:
     */
    if (prev->on_rq)
        update_curr(cfs_rq);

    ...
    if (prev->on_rq) {
        ...
        /* Put 'current' back into the tree. */
        __enqueue_entity(cfs_rq, prev);
    }
    cfs_rq->curr = NULL;
}
```

Listing 2.29: Reinserting the previous task

As I mentioned in the 2.5.2 section, the current task is considered to be on the runqueue, even if it was dequeued from it. It will be considered to be off the runqueue, if it entered in a sleeping state (as it reads in the kernel code, deactivated). So if the task is still running, `update_curr` is called, in order to calculate its new `vruntime`. Then it is inserted in the runqueue. Also note, that the `rb_leftmost` field is updated in the `__enqueue_entity` function.

```
static void __enqueue_entity(struct cfs_rq *cfs_rq, struct
    sched_entity *se)
{
    ...
    /*
     * Find the right place in the rbtree:
     */
    while (*link) {
        parent = *link;
        entry = rb_entry(parent, struct sched_entity, run_node);
        /*
         * We dont care about collisions. Nodes with
         * the same key stay together.
         */
        if (key < entity_key(cfs_rq, entry)) {
```

```
15              link = &parent->rb_left;
16          } else {
17              link = &parent->rb_right;
18              leftmost = 0;
19          }
20      }
21
22      /*
23       * Maintain a cache of leftmost tree entries (it is frequently
24       * used):
25       */
26      if (leftmost)
27          cfs_rq->rb_leftmost = &se->run_node;
28      ...
29  }
```

Listing 2.30: Leftmost task detection

### Wake up mechanism

The main function, again, is the `try_to_wake_up`, which does not do anything important (calculates some statistics, according to the waking up task's parameters and then activates it), apart from calling the `check_preempt_curr` function. This corresponds to the `check_preempt_wakeup` in CFS. As its name suggests, it checks whether the newly woken up task, is more suitable to run than the current running task.

```
1  static void check_preempt_wakeup(struct rq *rq, struct task_struct
       *p)
2  {
3      struct task_struct *curr = rq->curr;
4      struct cfs_rq *cfs_rq = task_cfs_rq(curr);
5      struct sched_entity *se = &curr->se, *pse = &p->se;
6      unsigned long gran;
7
8      if (unlikely(rt_prio(p->prio))) {
9          update_rq_clock(rq);
10         update_curr(cfs_rq);
11         resched_task(curr);
12         return;
13     }
14     /*
15      * Batch tasks do not preempt (their preemption is driven by
16      * the tick):
17      */
18     if (unlikely(p->policy == SCHED_BATCH))
19         return;
20
21     ...
22
23     gran = sysctl_sched_wakeup_granularity;
24     if (unlikely(se->load.weight != NICE_0_LOAD))
25         gran = calc_delta_fair(gran, &se->load);
26
27     if (pse->vruntime + gran < se->vruntime)
28         resched_task(curr);
```

```
29 }
```

Listing 2.31: Wake up preemption mechanism

First of all it checks whether the newly woken up task is of real time priority (`p` and `pse` represent the newly woken up task ). Real time priority tasks, will always preempt normal priority tasks, handled by CFS. Next, we can see that batch tasks, will never preempt another currently running task. Then finally, it moves onto doing some CFS stuff. `sysctl_sched_wakeup_granularity` is a predefined variable, set to 10 milliseconds, for tasks with nice priority number of 0 and it is used to prevent over-scheduling of a task, as mentioned in the kernel code [11]. If the task's priority is not of nice 0, then the wake up granularity is recalculated and if the sum of the calculated wake up granularity and the virtual run time of the woken up task is lower than the virtual run time of the current task, a preemption will occur.

# Chapter 3

# Methodology

Now that I discussed how both of them work, it is time to move on to the main goal of this thesis, the comparison of the aforementioned schedulers. I will try to cover both the performance of the two schedulers and their design where possible. Before I present my results, I need to describe the methodologies I followed, to draw them. So I will start with them.

## 3.1 The simulation mechanism

I wanted the performance comparison between CFS and O(1), to be as architecture free as possible, so I designed a simple simulation program in the C programming language. In a simulation environment, time can only be measured virtually and in the same manner, for all the components of the simulation and thus making the environment, architecture free. Also, there are no real time tasks involved, or hardware specific interrupts.

As far as the main program is concerned, I used the main functions of each scheduler, which I earlier discussed in the previous chapter, to implement the schedulers' logic. Of course they differ in some parts, because there was the need to implement the simulation's virtual clock, for example, but that did not come with the burden of not following the basic logic of each scheduler. Then in the main program, I implemented a very minimal operating system (actually only its scheduler counterpart), which referenced the modified main scheduler functions. As you will later note, I tried to create an algorithm as less stochastic as possible.

The simulation, uses a virtual clock. This clock is nothing more than a counter, that is increased using the scheduler tick function, which is called with 1000 HZ frequency (it is one of the default values in the kernel configuration file, for both schedulers), or every 1 millisecond. This does not mean that the program waits for 1 millisecond, before calling the tick function, but that every time this function is called, it will increase the clock by 1 millisecond. This means that, the current task will have been executed, for 1 millisecond.

The simulation's clock is implemented to increase each time, by a fixed value. So the simulation, is of fixed period of time type of simulation. The other (I did not implement it using this type) being the event driven simulation, which increases the clock by an amount, which is equal to the time period between now and the next event to occur (with preemption and wake up being the events, for example).

### 3.1.1 Simulation data and structures

As I did for the schedulers, I will start explaining the structures and the data I used for the simulation. I am not going to cover the scheduler functions I implemented, as their logic is the same as the ones I already discussed.

#### Input type structure

The simulation program, receives its parameters from the standard input. They are quite a few, so a type structure was needed. What follows, is this particular structure.

```
1  struct input {
2      int alg; // 0 for O(1) 1 for CFS
3      int test_type; // Task test (1), or time limit test (0)
4      long task_nr;
5      unsigned long long time_limit; // In secs
6      unsigned long long max_t_fin_time; // The maximum time required
       by tasks to finish.
7      unsigned int max_ints; // Max interrupts that can happen for
      each task
8
9      float interactive_percent;
10     float interruptible_percent; // 0 to 1. Number of interruptible
       tasks
11 };
```

Listing 3.1: The input structure

- `alg`: the scheduling algorithm to be simulated. 1 for CFS and 0 for O(1).

- `max_t_finish_time`: this is given in minutes. Each task will get a random value from the range [1, `max_t_finish_time`], which depicts the maximum time in milliseconds the task needs to run, before it exits.

- `time_limit`: the time limit, after which the simulation will stop.

- `test_type`: I implemented two types of tests. The first one, with option 0, issues a time limit (using the `time_limit` field), after which the simulation will stop, but it does not guarantee that the tasks will finish as well (or even run). The second, which is the one I will use to draw my results, runs the simulation, until all tasks have finished running, which means that their total execution time is greater or equal to the `max_t_finish_time` value (option 1).

- `task_nr`: the total number of tasks to be generated.

- `interactive_percent`: the percentage of tasks, that will be interactive. Only these tasks will get interrupted. The rest are considered as batch tasks and will never be interrupted.

- `interruptible_percent`: the percentage of times, an interactive task will become interruptible, when it receives an interrupt and go to sleep. Otherwise, it will be uninterruptible.

- `max_ints`: the maximum number of interrupts. Interactive tasks will get a random value from the range of [1, `max_ints`], which represents the number of interrupts, that it will receive, before it can exit. Each of these interrupts can happen in a random timestamp from the range of [0, `max_t_finish_time`]. I implemented this using a list, which I embedded in the task type structure. More on this later.

#### Constants

I used three constants. The one is for context switch time, which I set it equal to 0.01 milliseconds ([8]) and the other two for system time. I used two constants for system time, because it is different for the two schedulers. For O(1) I set it equal to 4.4 seconds and for CFS equal to 4.55 seconds ([3] their mean values), for each minute an interactive task has been running (the value scales according to the time the interactive task has been running). The system time of batch tasks was so small, I considered it irrelevant.

### 3.1.2 The simulation algorithm

The simulation algorithm for CFS follows the completely same logic for the O(1) simulation algorithm. I will present here the O(1) implementation, just for showing the logic I followed.

```
1 int main()
2 {
3     simulate();
4     return 0;
5 }
```

Listing 3.2: The main function

The main function, just calls the simulation function `simulate`, which actually handles everything.

```
1  void simulate()
2  {
3      struct input in;
4
5      get_input(&in);
6      file_init(in);
7
8      if(in.alg == 0)
9          simulate_o1(in);
10     else
```

```
11          simulate_cfs(in);
12  }
```

Listing 3.3: The simulation function

This one receives the input from the user and according to the algorithm selected, it creates some CSV files, for saving the statistics and passes the execution flow to one of the scheduler simulation functions, either `simulate_o1` or `simulate_cfs`.

```
1  // Take this as a simulated OS.
2  void simulate_o1(struct input in)
3  {
4      long wake_up_task = -1;
5      struct o1_task tasks[in.task_nr];
6
7      o1_rq_init();
8
9      for(int i = 0; i < in.task_nr; i++)
10         o1_new_task(
11             &tasks[i],
12             in.interactive_percent,
13             in.max_t_fin_time,
14             in.max_ints
15         );
```

Listing 3.4: The logic behind the simulation part 1

In this part, the algorithm, just initializes the runqueue (just initializes some pointers) of the corresponding algorithm (here the O(1) runqueue, which will be implemented by a list) and the tasks (random priority, number of interrupts etc). The `tasks` array, is an array of all the created tasks and the `wake_up_task` is the index of the array, containing the task, that will wake up next. Note here, that all tasks enter the runqueue at time 0.

```
1      while(1)
2      {
3          if(in.test_type == 0)
4              if(rq_o1.rq.clock >= in.time_limit)
5              {
6                  ivcs_stats_o1(in, tasks);
7                  break;
8              }
9          if(in.test_type == 1)
10             if(rq_o1.rq.task_nr == 0)
11             {
12                 ivcs_stats_o1(in, tasks);
13                 break;
14             }
15
16         o1_schedule();
```

Listing 3.5: The logic behind the simulation part 2

After the initialization, it enters in an endless `while` loop. The CPU will be idle at that time, so the `schedule` function is called in order to find the first, best

suitable to run next task. The if statements, are used to collect context switch statistics and also exit the loop, when the condition for each test type (which I discussed in the previous section) is met.

```
        while(!o1_need_resched)
        {
            // Preempt
            if(rq_o1.curr->ts.type
            && !list_empty(rq_o1.curr->ts.interrupt_times))
            {
                if(rq_o1.curr->ts.total_exec_time >=
                    list_get_first_int_timestamp(
                        rq_o1.curr->ts.interrupt_times))
                {
                    list_del(
                        list_get_first_node(
                            rq_o1.curr->ts.interrupt_times
                        )
                    );
                    o1_preempt_interrupt(in.interruptible_percent);
                    wake_up_task = o1_next_to_wake(
                                    tasks,
                                    in.task_nr
                                    );
                }
            }

            if(o1_need_resched)
                break;

            o1_scheduler_tick();

            // For statistics collection
            ...

             // Wake up conditions
            if(wake_up_task != -1)
                if(tasks[wake_up_task].ts.type == TASK_INTERACTIVE
                    && tasks[wake_up_task].task_array ==
                    rq_o1.sleeping)
                    if(rq_o1.rq.clock
                        > tasks[wake_up_task].ts.avg_wake_up
                        + tasks[wake_up_task].timestamp)
                    {
                        o1_wake_up_task(&tasks[wake_up_task]);
                        wake_up_task = o1_next_to_wake(
                                        tasks,
                                        in.task_nr
                                        );
                    }
        }
    }
}
```

Listing 3.6: The logic behind the simulation part 3

The second loop, is repeated until the current task is no longer suitable to continue running. Here the wake up, the tick and the preemption mechanisms can

take place:

- **preemption**: it can occur for interactive tasks and it is managed by the first `if` statement. If this statement is validated, then a check begins, to detect whether it is time for the current task to be preempted. This is done by checking if the total execution time of the task, is greater or equal to the next interrupt task execution timestamp (the interrupt timestamps, are saved in increasing order, in the `interrupt_times` list field, of each task). Then the node containing the interrupt timestamp, is deleted from the list and a preemption interrupt occurs (it directly calls the scheduler). Then it searches for the the task that will sooner wake up than the others, in the `tasks` array.

- **tick**: the tick function, gets executed, while the execution flow remains inside the second `while` loop.

- **wake up**: it is handled by the third if statement and again only interactive tasks can wake up. Like the preemption `if` statement, it is being checked at every tick. The embedded `if` statement, checks, whether the task at index `wake_up_task` must now be woken up, which will happen `avg_wake_up` milliseconds, after its last context switch, which was the one, after which it changed its state, to a sleeping state (the `sleeping` field, is a priority queue, that I implemented, which stores all sleeping tasks. For CFS this is done with `on_rq` field set to 0). Then it calls the main wake up function, which sets the flag for rescheduling (I called it `<alg>_need_resched`) to 1 and thus later exits the loop and searches for the next, sooner to wake up, task. As far as `avg_wake_up` is concerned, it can have a random value from the range of [1, `MAX_SLEEP_AVG`]. I used this range for both O(1) and CFS.

The CFS simulation function, just uses the same functions and variables, except that instead of the prefix `o1`, the prefix `cfs` is used.

## 3.2 Statistics and tests

Using the simulation program I made three types of testing:

1. The Batch testing: where all tasks are of batch type,

2. The Interactive testing: where all tasks are of interactive type and

3. The mixed testing: a percentage of tasks is interactive and the rest are batch.

### 3.2.1 The task tests

I run the simulation program a number of times for each of the tests, each time with different values. Also, because of the simulation's design, there is the need for a static seed, for the random number generator, so as for the program to produce tasks, with the same characteristics each time it is being run, with a different scheduler, for the

same test.

#### The batch test

The batch test is the most simple one. To begin with, I created only one task, in the runqueue. Then, I run the simulation 10 times, each time increasing the task number by 100, starting from 100 tasks. I kept the maximum finish time of the tasks to two minutes, for all these tests.

#### The interactive test

Again, I made a test which, consisted of only one interactive task, with a maximum of 100 interrupts and a maximum finish time of 15 minutes. Here I made and some middle tests, consisting of 100, 200, 300, 400 and 500 interactive tasks, each, with up to 1000 interrupts and a finish time of 10 minutes. After the previous set of tests, I made another one set, where I rapidly increased the maximum number of interrupts to 1 million and reduced the number of tasks to 10, 20, 30, 40, 50, for each test, and their maximum finish time to 5 minutes. All of the tests, as I described, are consisted of only interactive tasks, which means, that there is the need to define the percentage of the times each task, will become interruptible. For this reason, each of the aforementioned test sets, were run 3 times in total, with different interruptible percentage each time, with these values being 20, 50 and 80 percent respectively.

#### The mixed test

I implemented some tests, each time with a different value for the max interrupts, the task number and the max finish time, while keeping the interruptible percentage to 80 percent, because uninterruptible tasks are more rare than the interruptible and as you will note in the results of the interactive test, changes in this field, resulted in minor performance downgrades. The following test sets, were run 3 times each. Every time I run them I changed the interactive percentage of the tasks starting from 20 percent, then 50 and finally 80, with the rest just being batch tasks. In total two test sets were created. In the first one I tested the performance of each scheduler for 100, 200, 300, 400 and 500 tasks, with a maximum finish time of 15 minutes each and 1000 maximum interrupts. The final test, tested the two schedulers for 50, 100, 150, 200 and 250 tasks, with a maximum finish time of 10 minutes and 1000000 maximum interrupts.

### 3.2.2 The statistics

I am going to compare the schedulers based on several statistics, which are the established statistics used, for scheduler comparisons. These are the elapsed time of a task and its standard deviation (for fairness measurements), the number of involuntary context switches of each task and the system time. In most cases, these values, will appear as the average of all tasks participated in a test of a test set. The voluntary context switches, are task dependent and have nothing to do with the scheduler performance comparison. The way they handle the interrupts, hence the voluntary context switches and the interactive tasks, as well as the time slice

for every task, on the other hand will have consequences on their performance. All of the statistics, are extracted from the simulation program into CSV files.

# Chapter 4

# Test results

The chapter, will be divided into three sections, each of them discussing the results of a different type of testing. They will be listed in the order they were described in the previous chapter. So I will begin the Batch test, followed by the fully Interactive test and last will be the Mixed task test.

## 4.1 The Batch task test

### 4.1.1 Single batch task

For only one batch task, there is no need for a diagram. The number of involuntary context switches is 0, the elapsed time is equal to the task's finish time (12.78 seconds) and the system time is equal to 0, for both schedulers.

### 4.1.2 Multiple batch tasks

I will begin with the elapsed time. Figure 4.1, shows the average elapsed time for all task tests conducted (100, 200, 300, ..., 1000 tasks), all with a maximum finish time of 2 minutes.

In the aforementioned figure, we can see, that by increasing the number of tasks, no matter their priority, or their finish time, the average elapsed time of the tasks will increase monotonically. This incrementation has a more drastical effect on O(1) than on CFS. You can also note, that after the amount of 700 tasks, the effect on the average elapsed time of tasks, scheduled by CFS, slightly decreases. The differences between the two schedulers, on the other hand are very minimal.

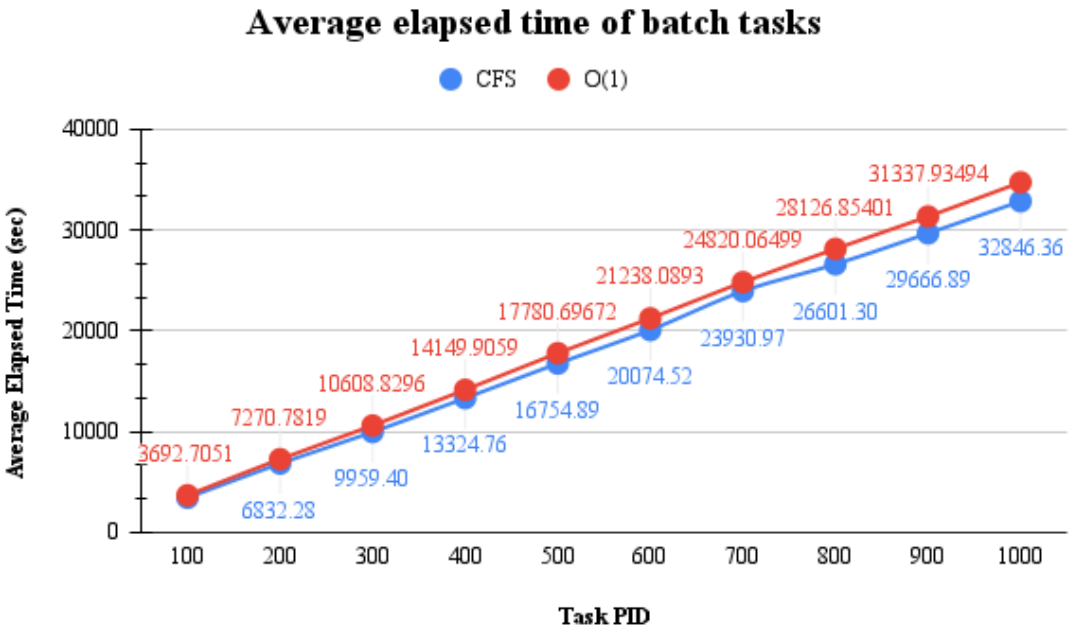


Figure 4.1: The average elapsed time of the batch task test

The standard deviation, of the elapsed times, can be viewed as a measure of the fairness of each algorithm. According to figure 4.2, CFS and O(1) have small differences in standard deviation of elapsed times. This means that, for batch tasks, at least, they behave in the same fair way. It is noteworthy to mention, that O(1) has smaller values of standard deviation, than CFS, making it a bit more fair in batch scheduling, than CFS.

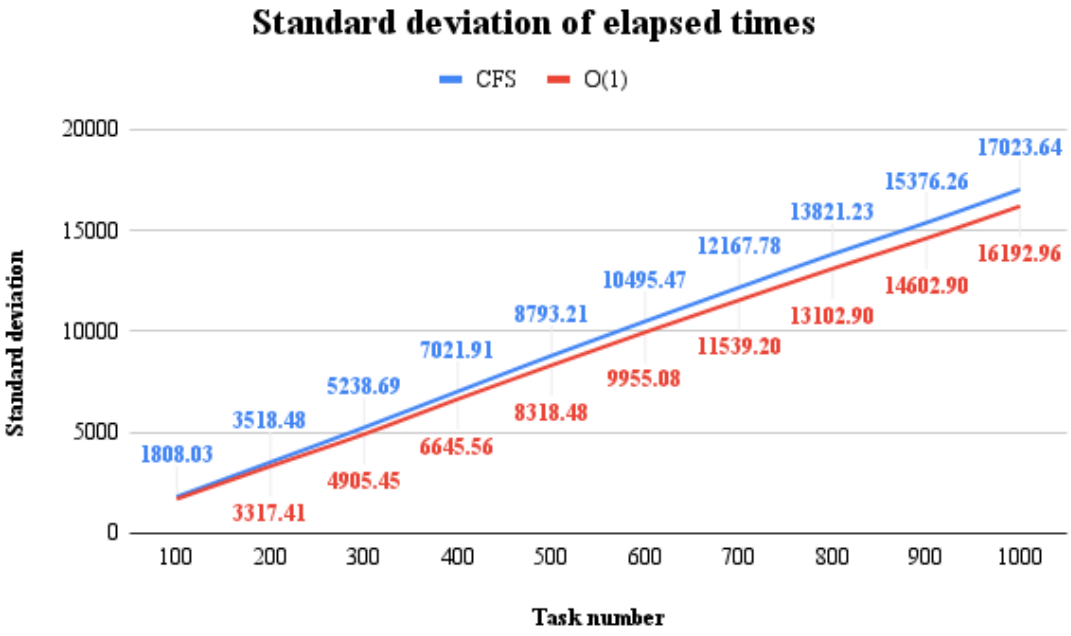


Figure 4.2: The standard deviation of elapsed times (batch task test)

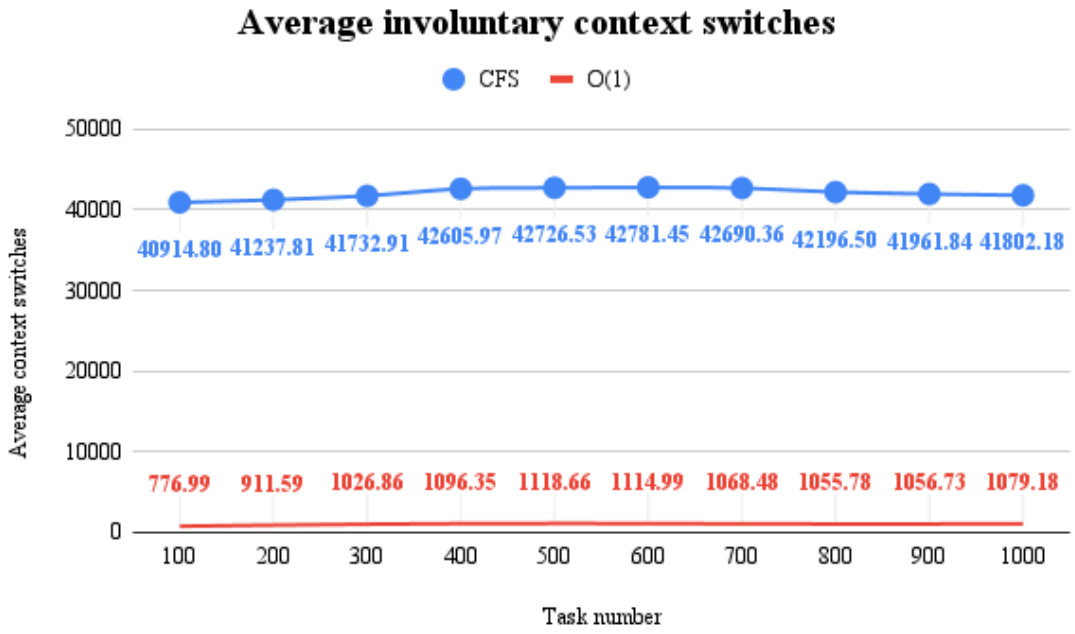


Figure 4.3: Average involuntary context switch (batch task test)

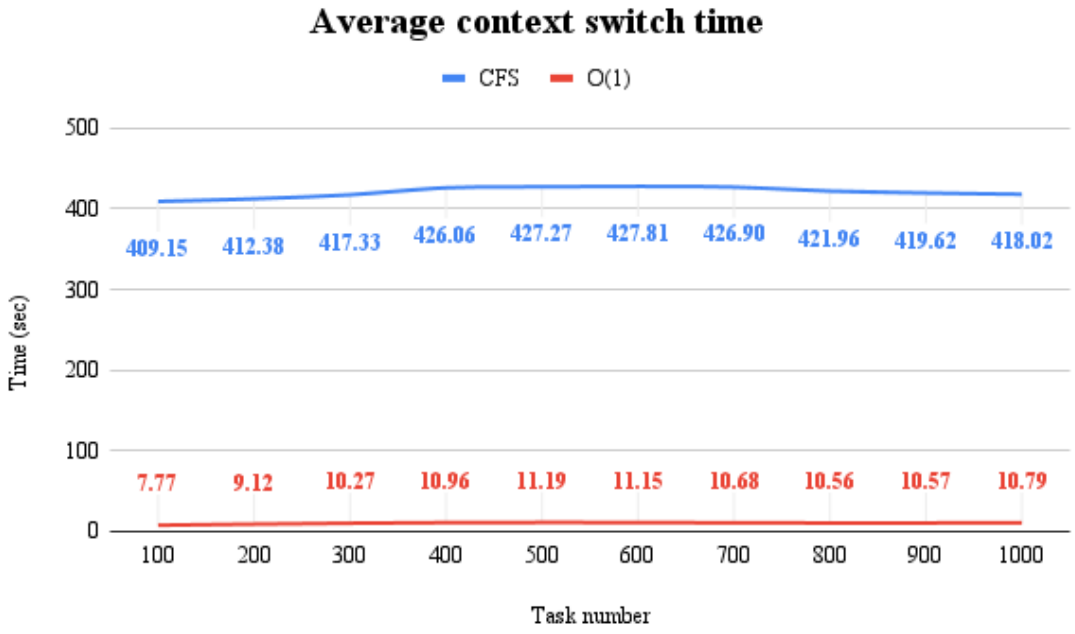


Figure 4.4: Average involuntary context switch time (batch task test)

Figure 4.3, shows the average involuntary context switches, of the tasks, scheduled by each of the two schedulers. Notice that the average value, in theory, remains the same (the changes are so small, that can actually be considered irrelevant) as the number of batch tasks, that need to be scheduled increases. A heavy amount of involuntary context switches, occured in CFS, compared to the amount occurred in O(1). Combining this with the results from figure 4.1, we can conclude that, CFS achieves the similar amounts of elapsed time as the O(1) scheduler (a bit better

in practice) while, also, it significantly increases the number of involuntary context switches taking place.

Figure 4.4, shows, the average time, that each scheduler spends at context switching tasks. This refers to involuntary context switch (remember that in the simulation program, context switch time is equal to 0.01 milliseconds).

The high number of involuntary context switches in CFS, is attributed to the very small time slice each task receives, compared to the ones issued by the O(1). This causes more task preemptions, hence the high values of context switching.

## 4.2 The Interactive task test

### 4.2.1 Single interactive task

Running a single interactive task with both schedulers resulted in the same data for both an interruptible and an uninterruptible task, for the corresponding scheduler. The elapsed time was 186.69 seconds for CFS and 186.34 seconds for O(1), the system time is equal to 10.38 seconds and 10.04 seconds accordingly, while the involuntary context switches are 0. With these first results, we can already see that the CPU spends much of its time executing system code, when an interactive task is running.

### 4.2.2 Multiple interactive tasks: small amount of interrupts

This section will be covered in 3 different parts, each concerning a different test set, according to how they were specified in chapter 3.

**20 percent interruptible test set**

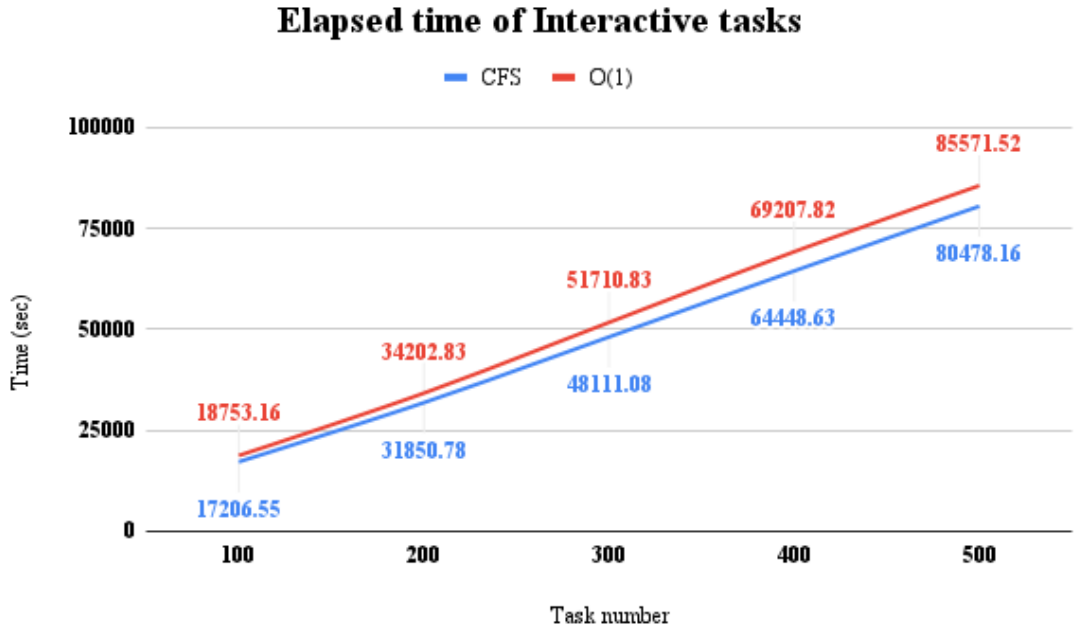


Figure 4.5: Average elapsed time for the first test set

Interactive tasks, which become interruptible 20 percent of the total times they enter a sleeping state, have better elapsed times when handled by CFS, than O(1). The differences are relatively small though, but become more noticeable, while increasing the number of running interactive tasks.

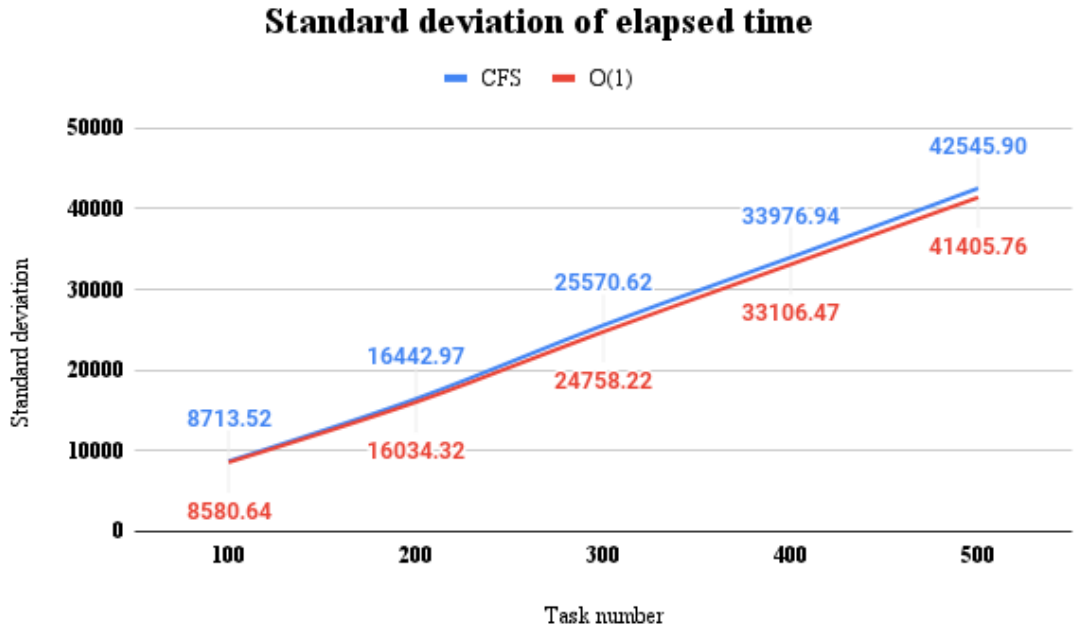


Figure 4.6: Standard deviation of elapsed time for the first set

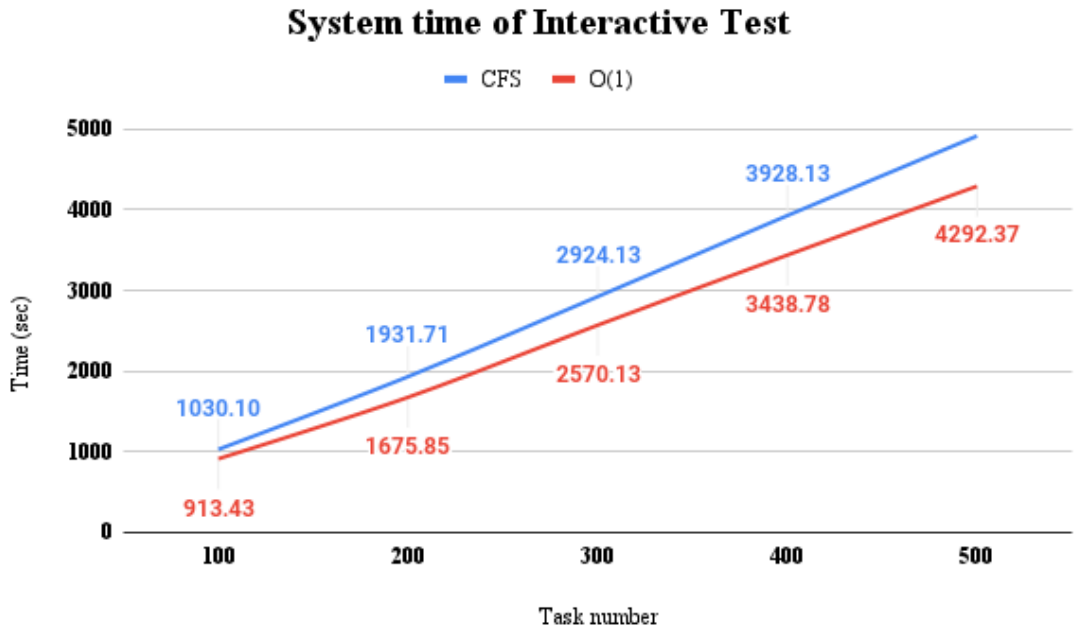


Figure 4.7: Average system time of the first set

The standard deviation of elapsed times, as shown in figure 4.6, is monotonically increasing for the two schedulers, reducing their fairness, as the number of interactive tasks, with different and random priorities increases. The differences, on the other hand between standard deviation of CFS and O(1) are very small, so in regard of how fair they behave, it can be noted, that they behave on the same fair way for the aforementioned type of tasks.

By examining figure 4.7, we can see that O(1) has smaller system time values, than CFS (note that the fifth value from the CFS line is missing). Keep this in mind for now, as it will prove useful, while examining the next figure.

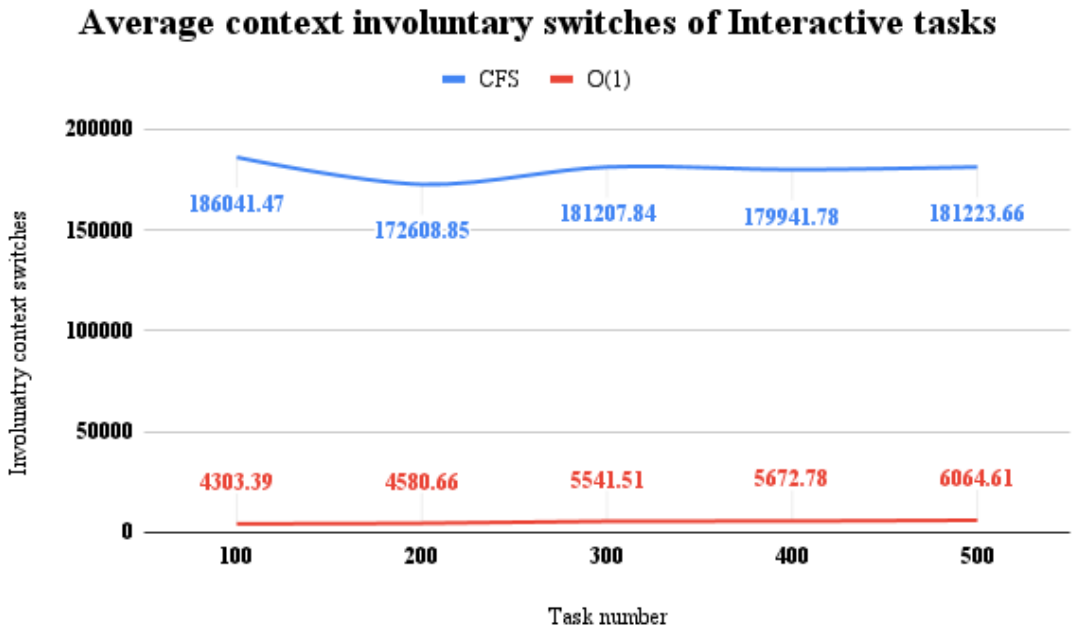


Figure 4.8: Average involuntary context switches of the first set

Figure 4.8, shows the average context switches regarding each of the tests, for this test set, for both schedulers. O(1), as it seems, outmatches CFS in context switches. On the other hand these average values remain steady, as the number of interactive tasks increases. Now combine these results, with the ones shown in the previous figures and remember that context switch time was defined to 0.01 milliseconds, which means that O(1) has, of course, also, lower context switch time values. So O(1), has lower context switch time values and system time values and also is equally fair to CFS, when scheduling this type of interactive tasks, but has higher elapsed time values than CFS. With these in mind we can make a first assumption, that with O(1) the CPU may remain idle for bigger time periods and thus resulting in smaller amounts of CPU usage, than in CFS. This may explain, why CFS has lower average elapsed time, while higher context switch and system time than O(1). This may also mean, on the other hand, that some tasks monopolize the CPU, or even both may be happening at the same time. Whatever the case, this seems to be an issue for the O(1) algorithm.

### 50 percent interruptible test set

As you will note, not much has changed when comparing this set with the previous one. O(1) still has higher average elapsed times, than CFS, while lower values of average system time and involuntary context switches.

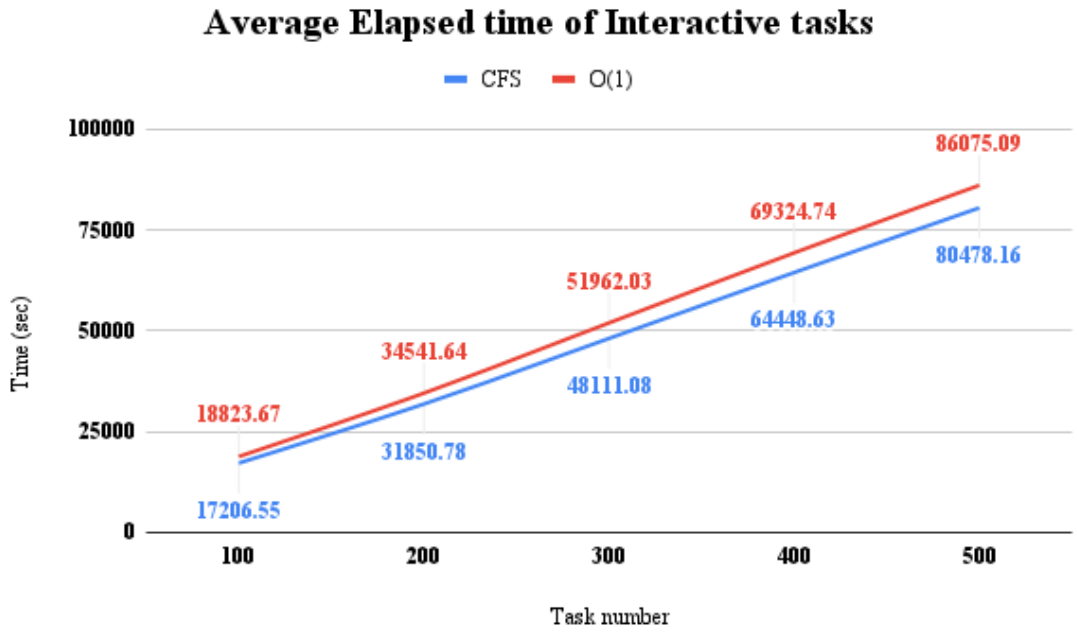


Figure 4.9: Average elapsed time of the second test set

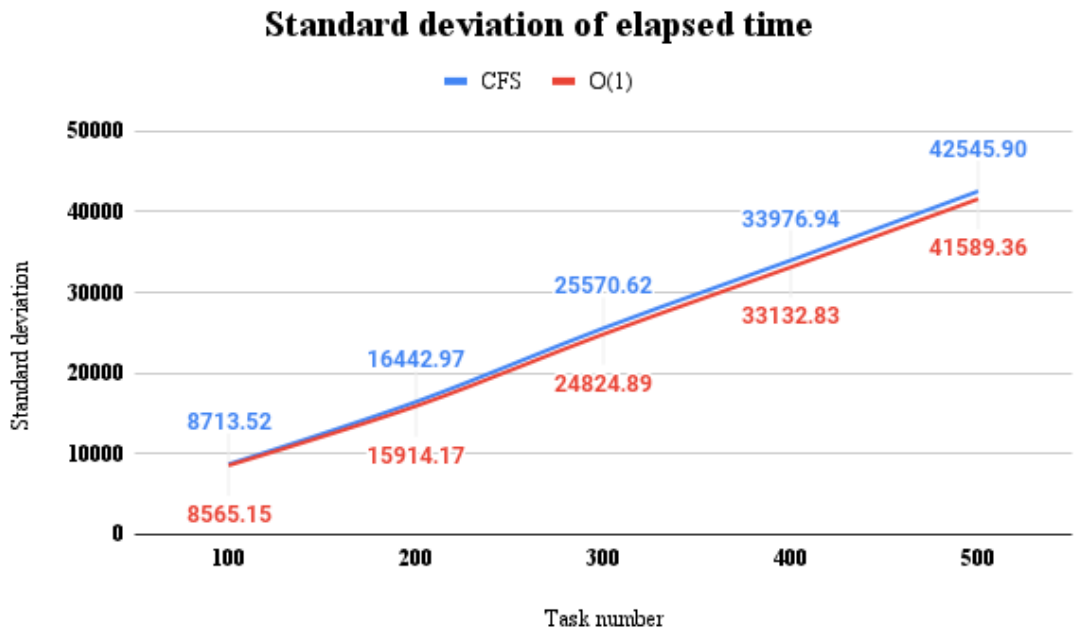


Figure 4.10: Standard deviation of elapsed time of the second test set

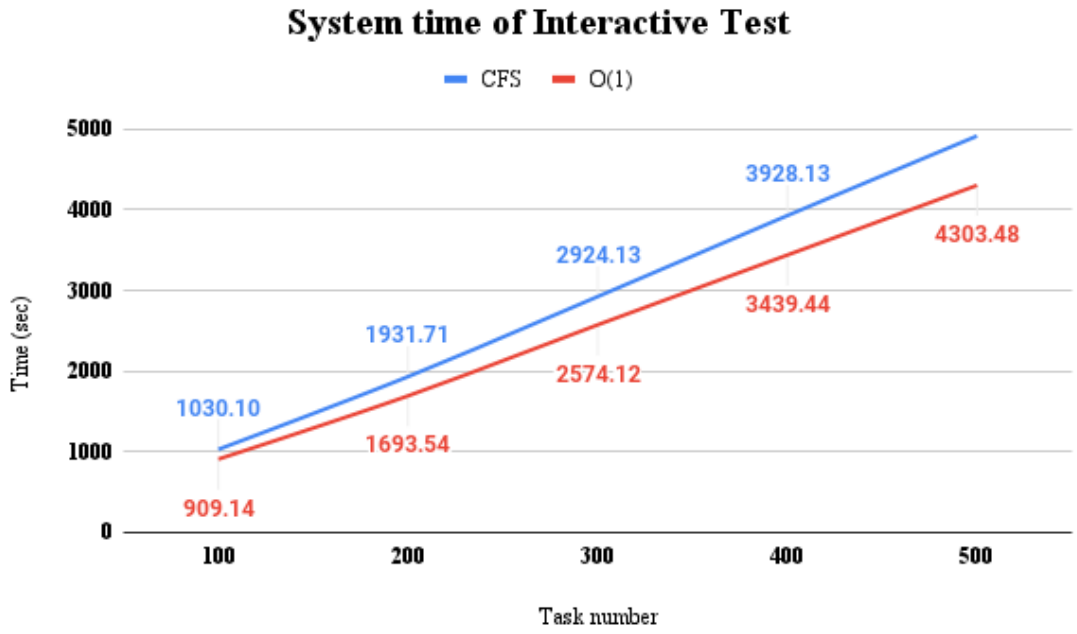


Figure 4.11: Average system time of the second test set

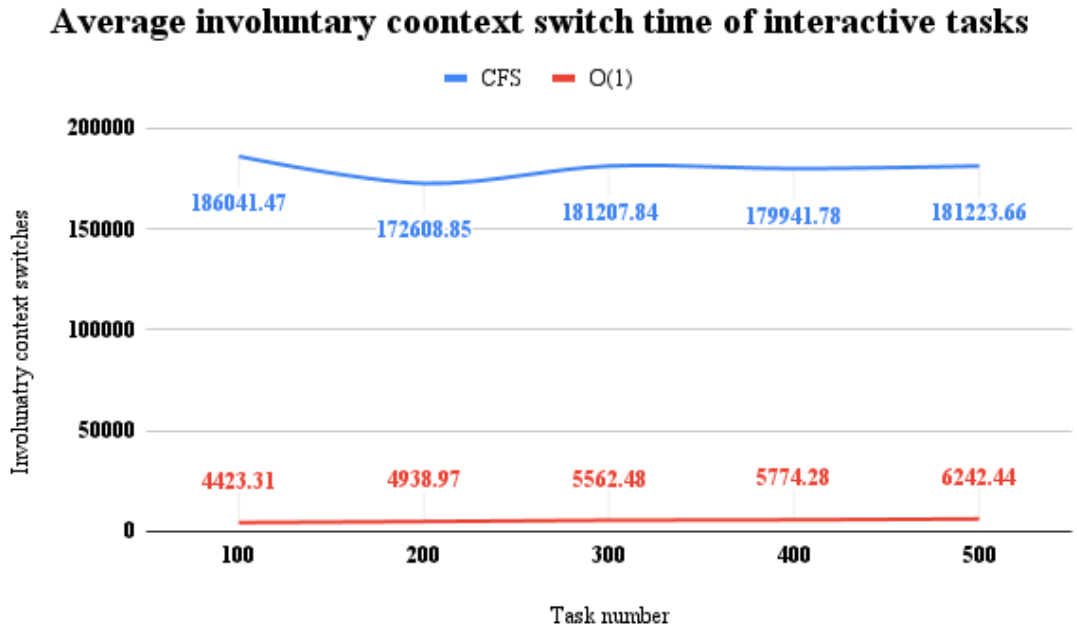


Figure 4.12: Average involuntary context switches time of the second test set

Other than that, the aforementioned values do not dramatically change, when increasing the number of the interruptible percentage field. So the schedulers' behaviour, remains the same as before.

### 80 percent interruptible test set

The final test set for interactive tasks, will provide the final conclusion to the assumptions previously made. The assumption, regarding the idle time of O(1), will

only be partially answered in this part.

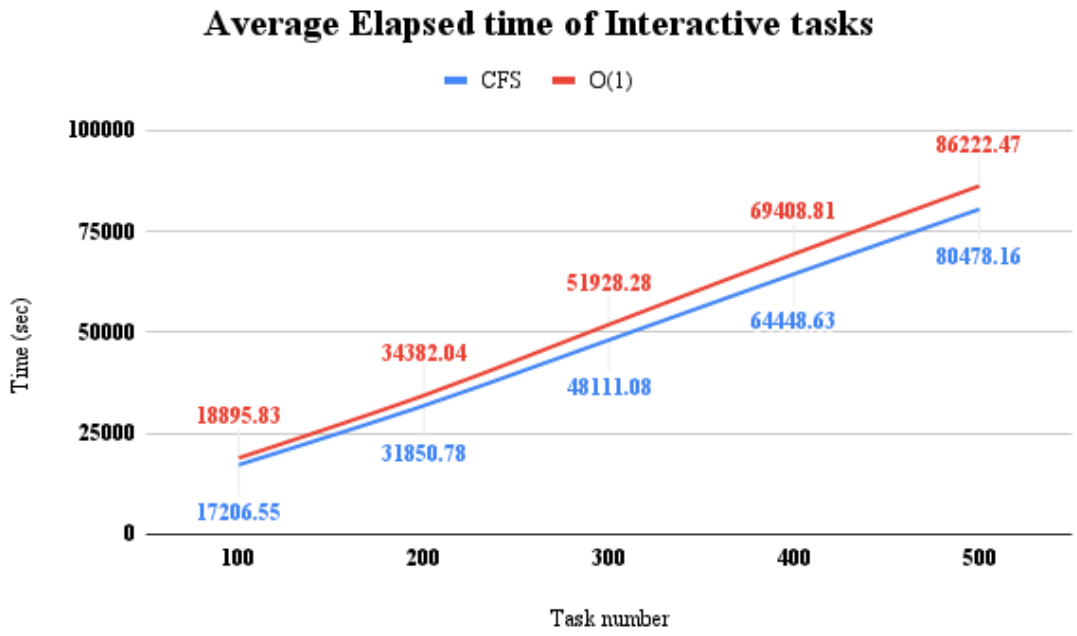


Figure 4.13: Average elapsed time of the third test set

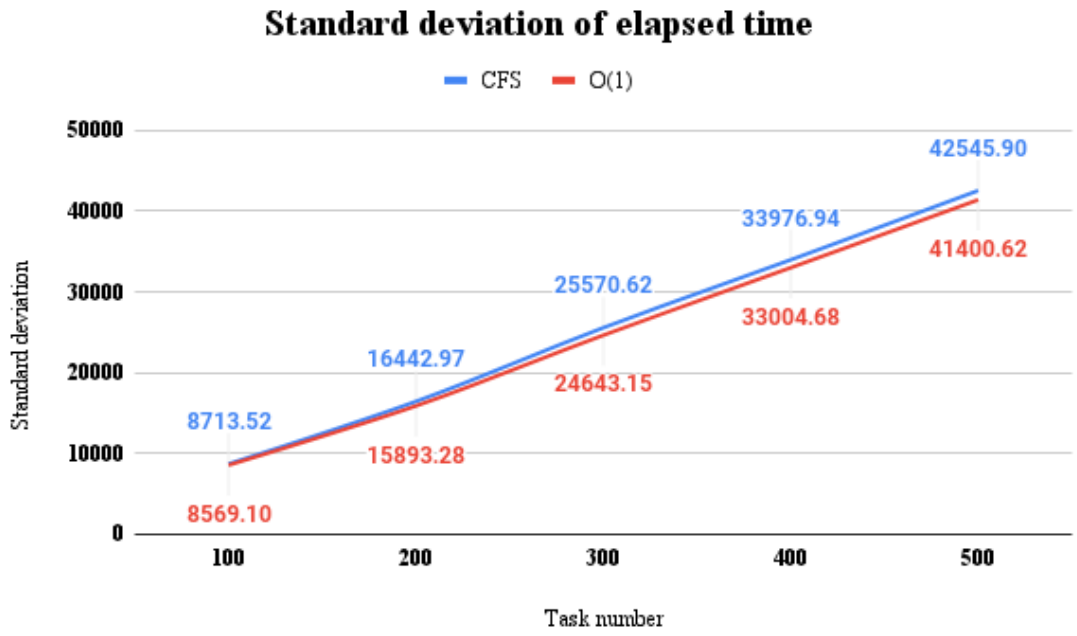


Figure 4.14: Standard deviation of elapsed time of the third test set

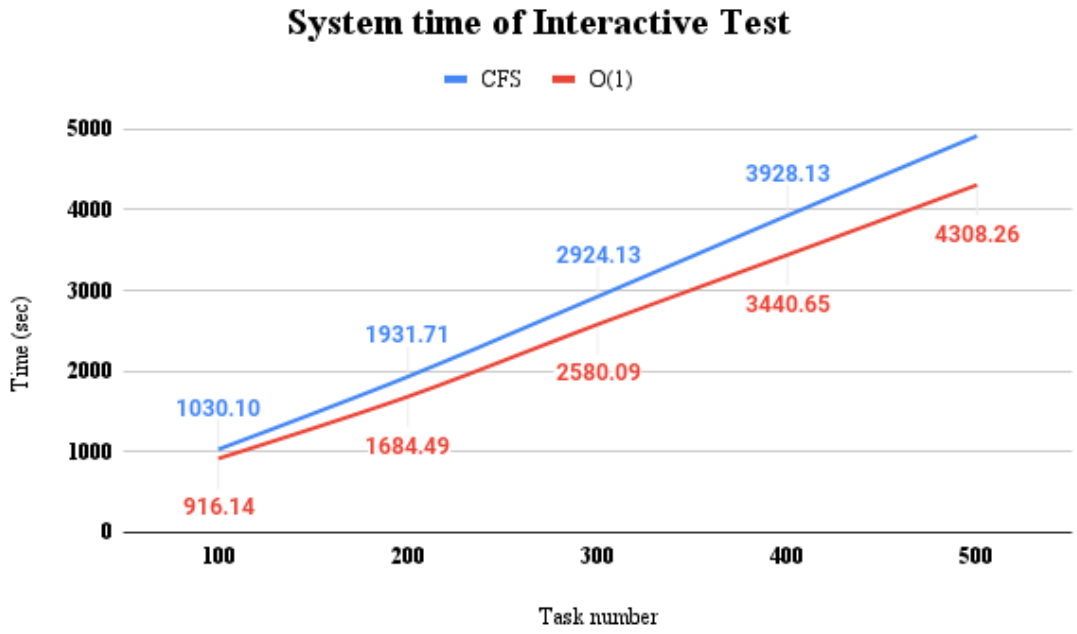


Figure 4.15: Average system time of the third test set

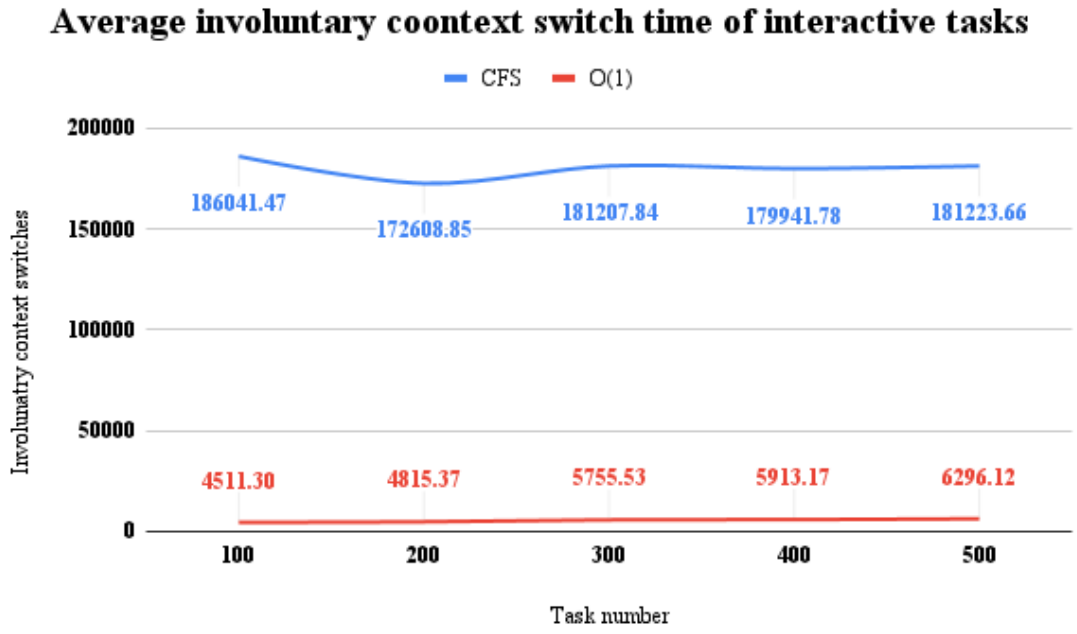


Figure 4.16: Average involuntary context switches of the third test set

Inspecting the diagrams, shown in the 4 above figures, we can draw the conclusion, that the two algorithms' behaviour is not dependent on the number of times a task is considered uninterruptible or not. Also the assumption made about the idle time and the CPU monopolization in O(1), can actually be verified, at least for interactive tasks with a small amount of interrupts (maximum 1000). This problem, also deepens, as the number of running interactive tasks increases, while taking into consideration, that the average number of involuntary context switches remains the

same (while increasing the number of running interactive tasks) and the average elapsed and system times increase.

### 4.2.3 Multiple interactive tasks: huge amount of interrupts

In the previous section (4.2.2), it was made clear, that the behaviour of the two algorithms remains the same, while the number of times an interactive task can become interruptible, increases. CFS outmatched O(1) in the previous set of test sets and some O(1) issues were uncovered. Now, it remains to examine, what happens when tasks with a huge number of maximum interrupts are waiting to be executed, to ultimately verify these assumptions. The results from the following test sets, may be a bit more interesting.

#### 20 percent interruptible test set

Figure 4.17, shows the average elapsed time of the tasks, participating in the first set, of this test set. While both schedulers start from the same point, CFS manages, to keep the average elapsed time at lower levels than O(1), while the number of interactive tasks increases. Although O(1), manages to stabilize the elapsed time, at some point, its value is still 10 times larger, than that of CFS for the same number of interactive tasks.

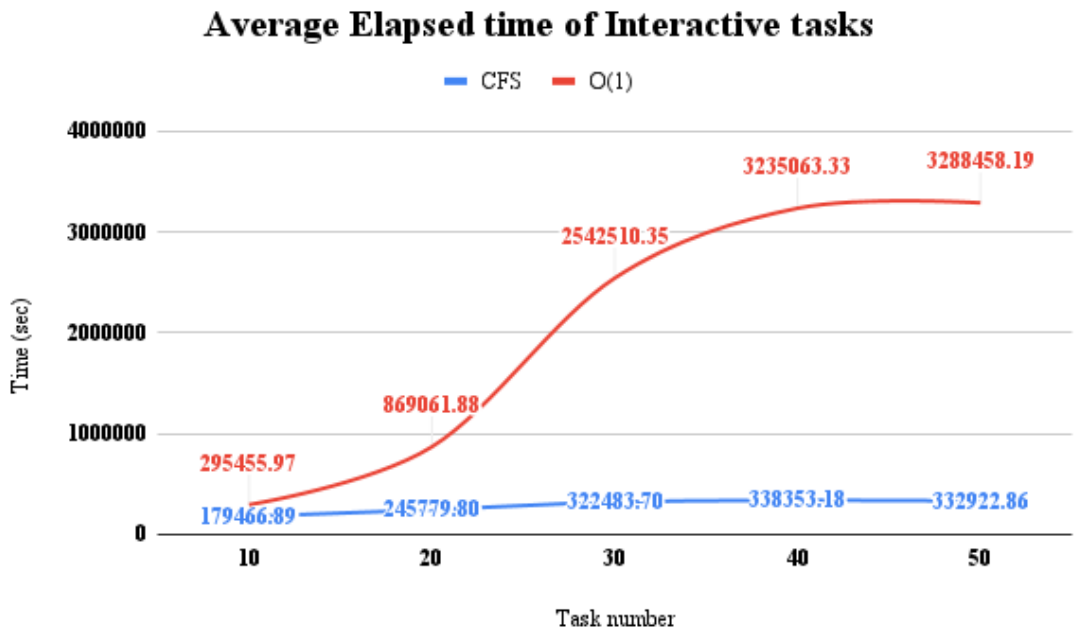


Figure 4.17: Average elapsed time of the first test set

The below diagram (4.18) sees, O(1) loose in terms of fairness regarding scheduling interactive tasks, with a huge amount of interrupts. As the number of these tasks increases, so does the unfairness of O(1) in scheduling them. On the other hand, CFS manages, not only to achieve better fairness levels than O(1), but also keep them steady, as the number of this type of tasks increases.

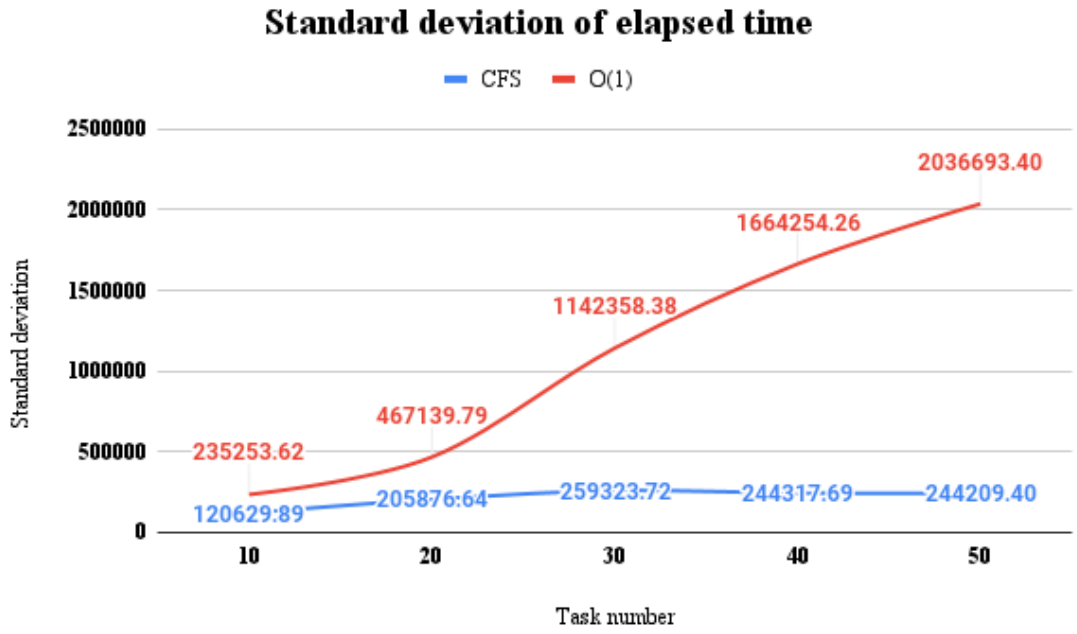


Figure 4.18: Standard deviation of elapsed time of the first test set

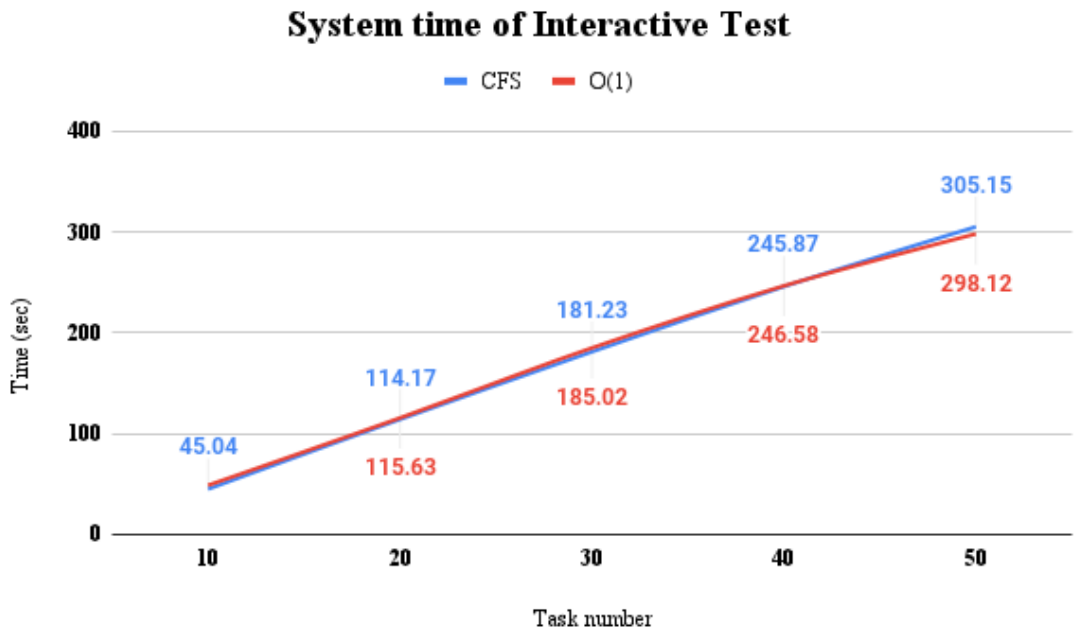


Figure 4.19: Average system time of the first test set

Other than that, CFS also manages to compete with O(1) in system times, as shown in the diagram above (4.19). The difference, between the average system time values of each scheduler, for the corresponding amount of tasks, remains very small and nearly steady as the number of tasks increases.

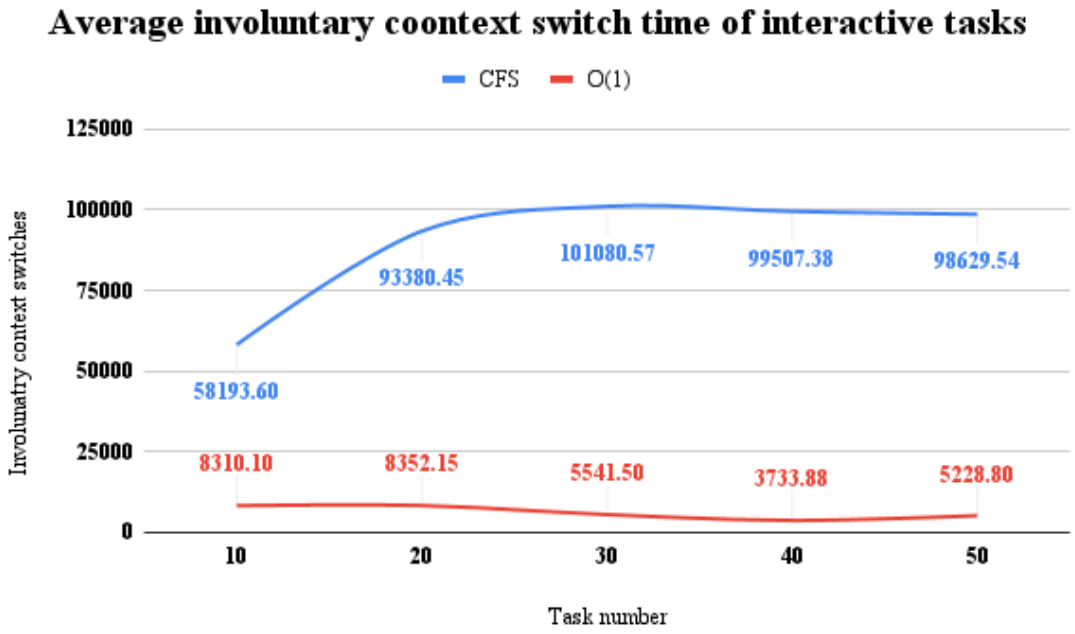


Figure 4.20: Average involuntary context switches of the first test set

O(1), on the other hand, still manages involuntary context switches better than CFS, which as discussed in 4.2.2 section, seems interesting. But before drawing any final conclusions, there are two more test sets to be conducted.

### 50 percent interruptible test set

Now I will examine, if by increasing the number of interruptible percentage chances, has any major effects in each of the schedulers' behaviour.

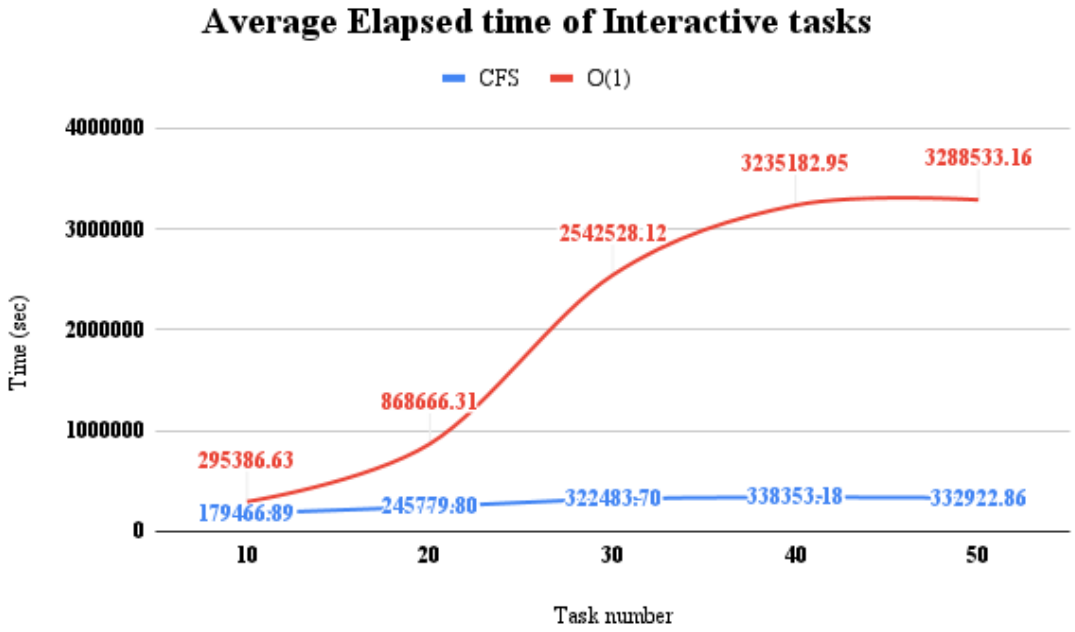


Figure 4.21: Average elapsed time of the second test set

This time the average elapsed time increases, for each test. What interests us, though, mostly, is the way it increases, while the number of interactive tasks increases. By examining the diagrams 4.21 and 4.17, the only differences that we can detect are the values. The way they increase, remains the same.

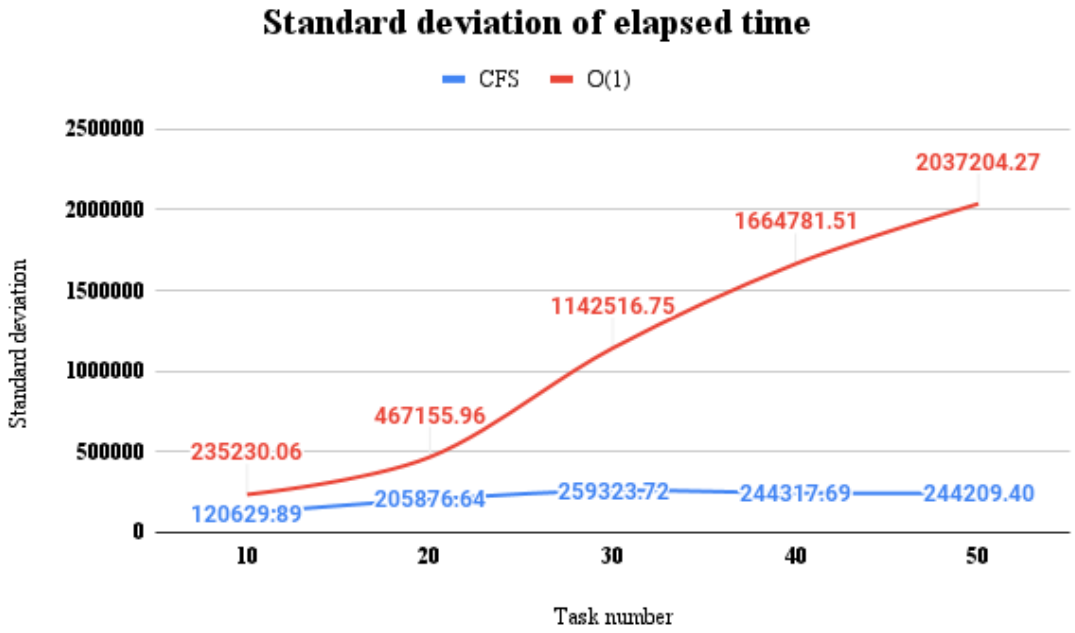


Figure 4.22: Standard deviation of elapsed time of the second test set

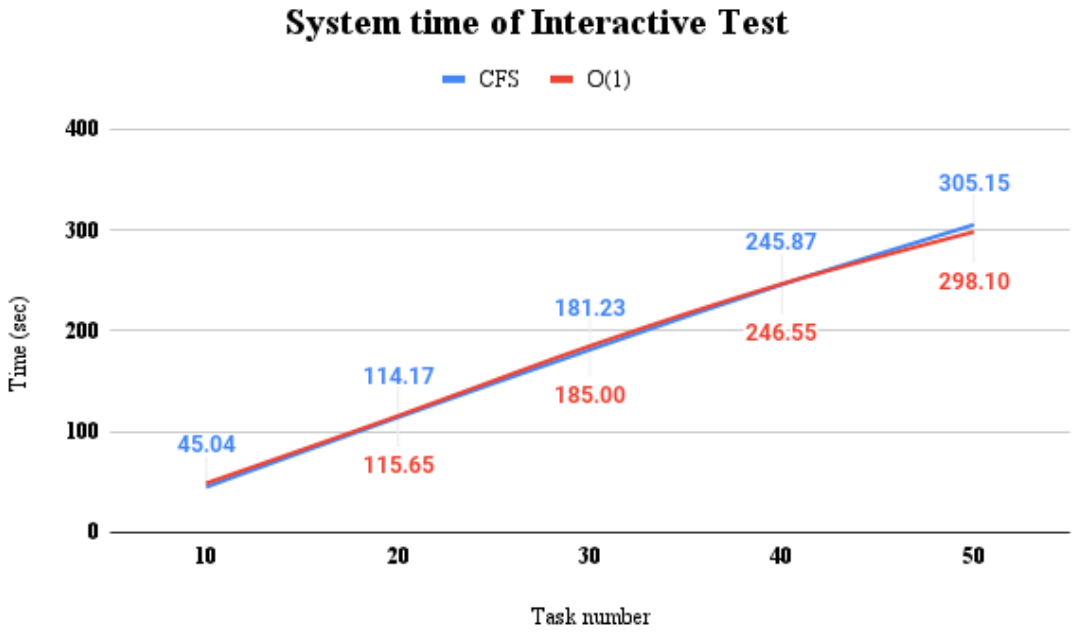


Figure 4.23: Average system time of the second test set

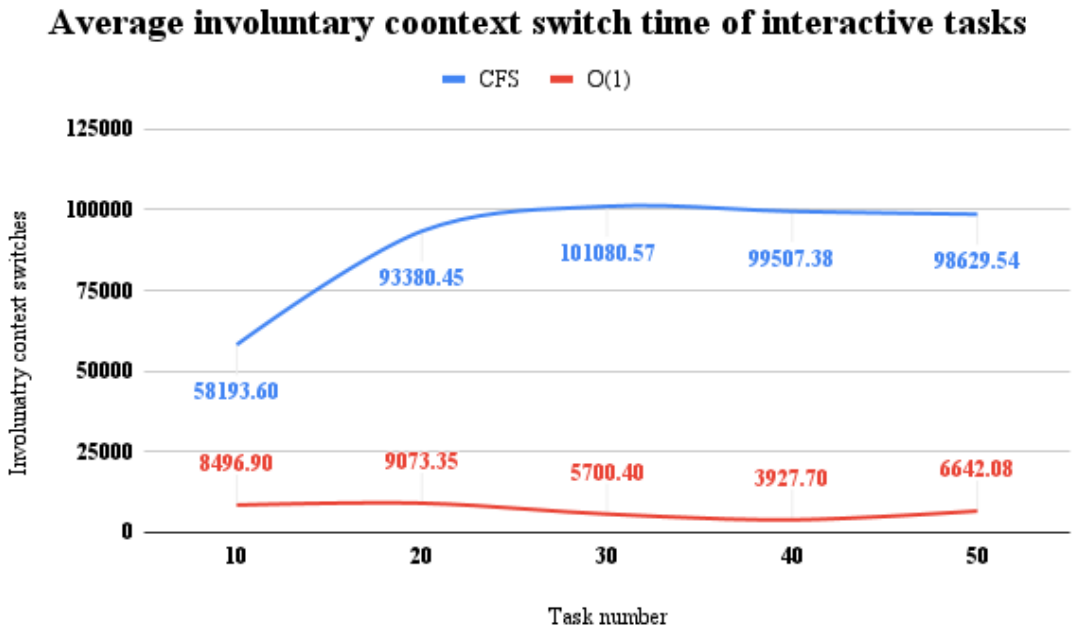


Figure 4.24: Average involuntary context switches of the second test set

The same happens with the three above diagrams, when comparing them with their counterparts in the previously shown test set.

### 80 percent interruptible test set

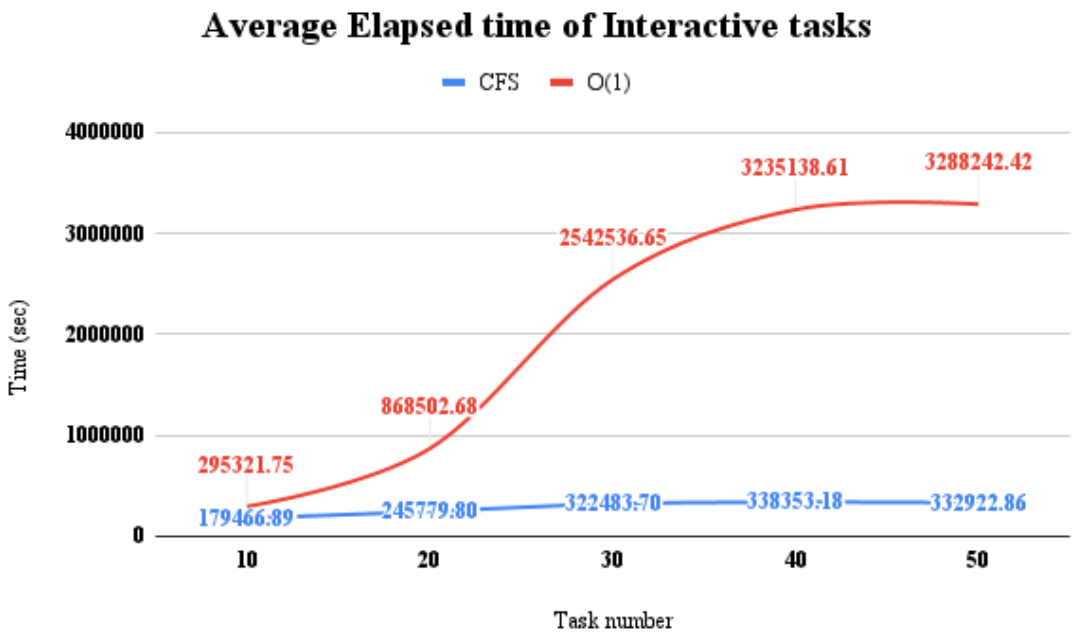


Figure 4.25: Average elapsed time of the third test set

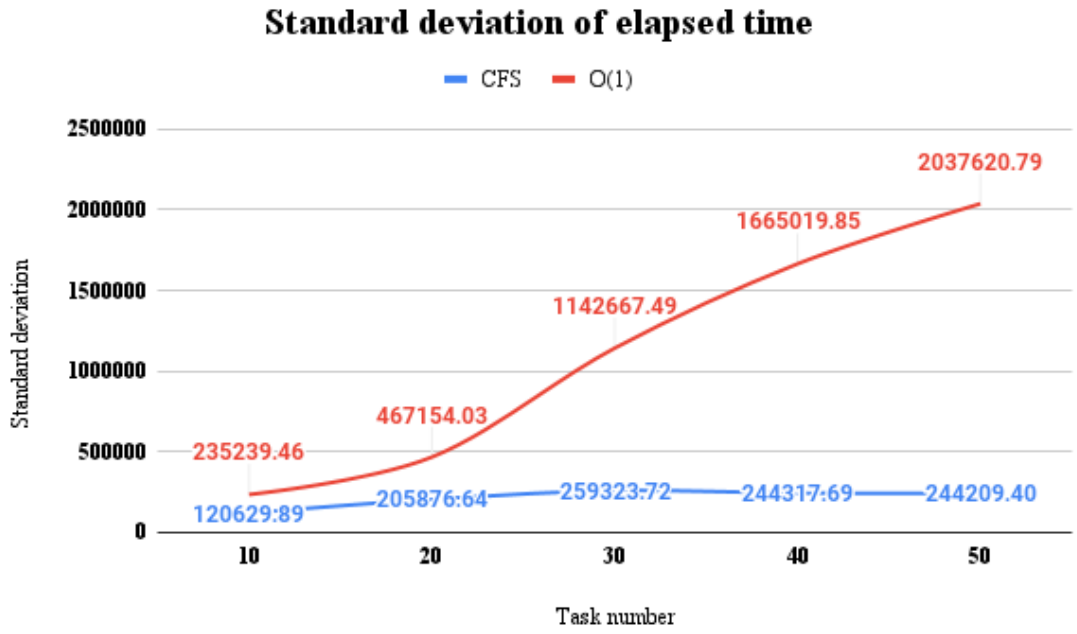


Figure 4.26: Standard deviation of elapsed time of the third test set

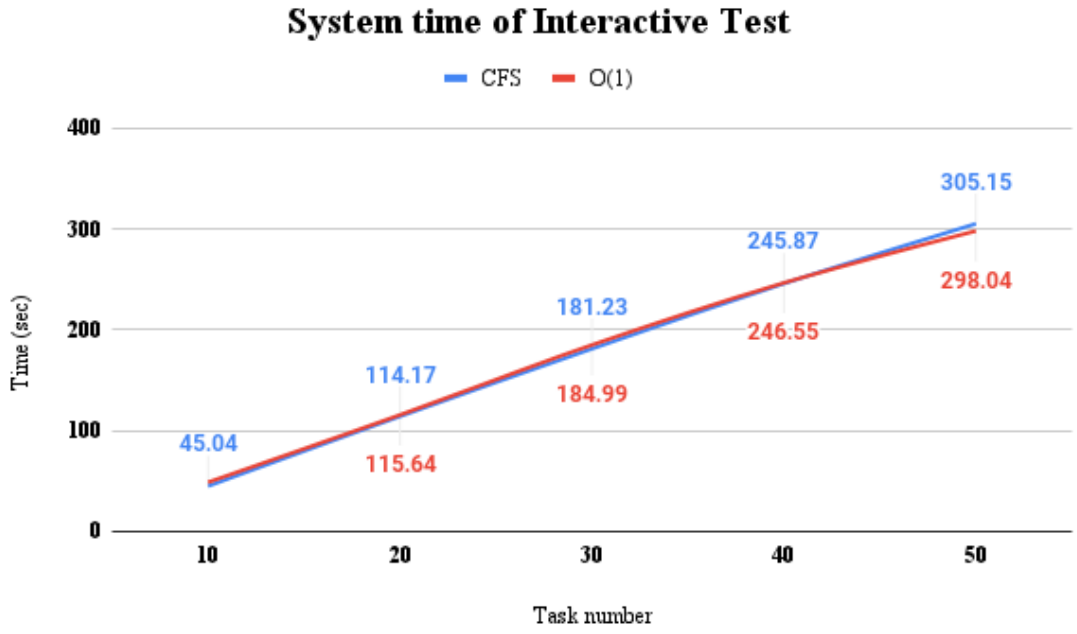


Figure 4.27: Average system time of the third test set

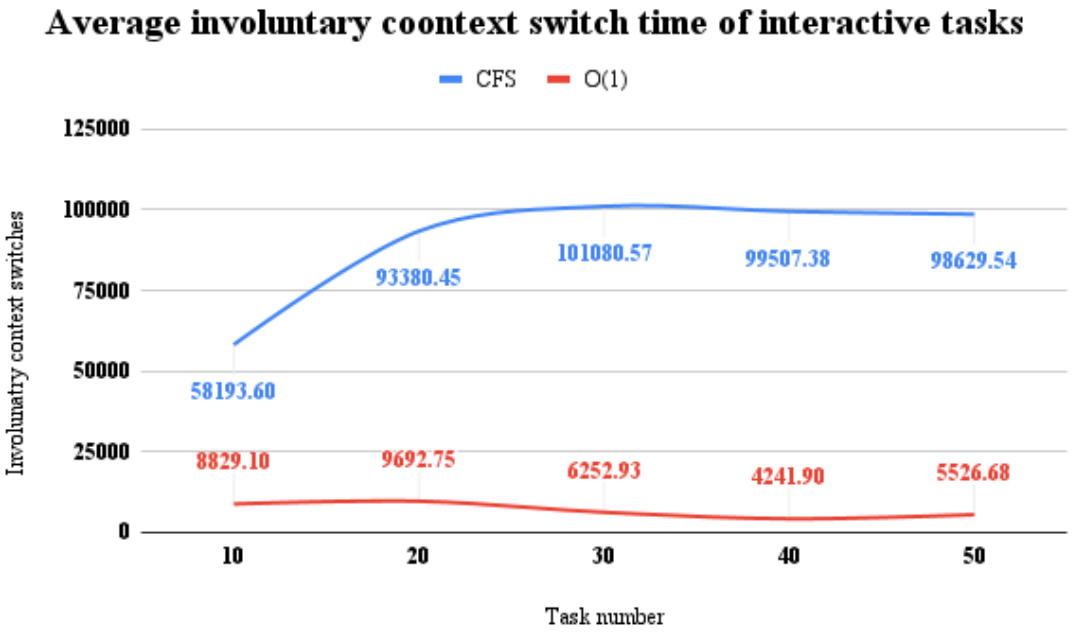


Figure 4.28: Average involuntary context switches of the third test set

Comparing the 4 above diagrams, with the ones from the previous 2 test sets, described in this section, no difference can be found (except of course for the differences in the diagram values). Everything, that was mentioned in the first test set in this section, also implies here.

Not only this, but another interesting thing is that again, O(1), while having lower values of context switches and theoretically the same values of system time, will finish, much later than CFS. So, this assumption can finally be validated, at least for interactive only scheduling.

### 4.2.4 Summary of the Interactive test

The six sets of test sets, that where described in sections 4.2.2 and 4.2.3 lead to some noteworthy conclusions. The first one being that, the amount of times an interactive task can become interruptible, does not influence the behaviour of the two schedulers. Of course, the average values will increase, but this incrementation is actually the same for both O(1) and CFS. So in overall nothing really changes.

The second and maybe, most important of the two conclusions, is that O(1) fails to solve the CPU monopolization problem or achieves very low CPU utilization or perhaps both, when scheduling interactive tasks. Other than that, it was shown, that O(1), was less fair than CFS in this type of scheduling.

Why the monopolization problem occurs in O(1), can be explained easily. Remember, that O(1) gives a priority boost, to tasks, according to their `sleep_avg` value, especially for interactive tasks. This is also designed to decrease slowly, meaning that under certain circumstances, many tasks may receive the highest normal

priority (100) and the tasks will be executed in a FIFO order, but with a time slice enabled for each one. Imagine, for example, that most of the tasks managed to achieve this high priority, but, some others did not and are at the bottom of the priority queue (140), but we need to run one of them. This task, will have to wait for the highest priority ones to finish their time slice and then run. What will happen, if the highest priority tasks become trapped in a huge cycle of sleeping and running again, without finishing their time slice, which could last up to 5 minutes? They will continuously get bonuses, which will start decreasing of course after some point, but it will take some time to reach the lowest possible bonus.

## 4.3 The Mixed task test

I will start explaining the results of the first mixed tasks, set of test sets and then move on to the second one.

### 4.3.1 Mixed tasks: small amount of interrupts

#### 20 percent interactive tasks

20 percent of the tasks from each set, of the test set, are interactive tasks and the rest are batch tasks. Beginning with the elapsed time we can see that the difference between O(1) and CFS's elapsed time, increases, as the number of total tasks increases. CFS has the smallest average elapsed time of the two schedulers.

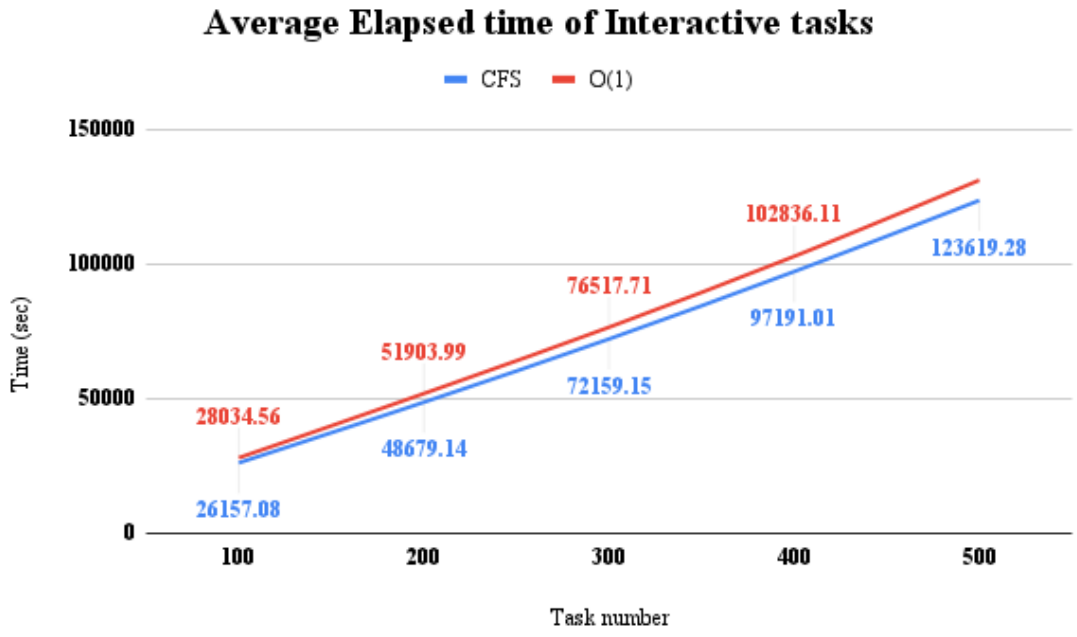


Figure 4.29: Average elapsed time for the first test set

As shown in figure 4.30, they are equal in fairness, at least when the percentage of interactive tasks is small and their fairness is reduced, as the number of tasks increases. Also in figure 4.31, we can see that, still O(1), has a lower amount of

average system time than CFS.

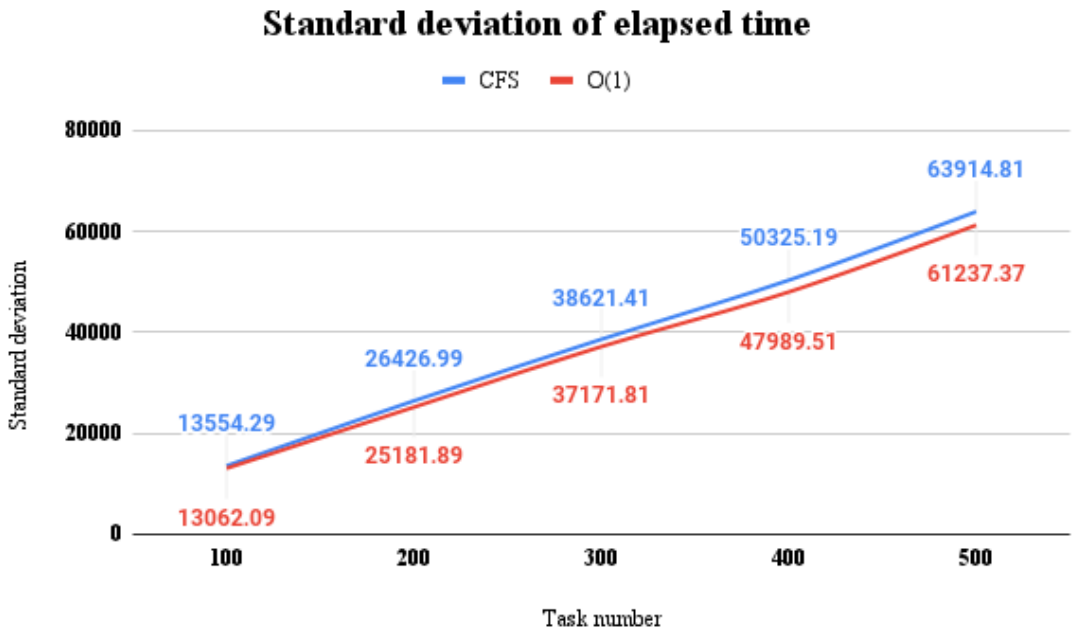


Figure 4.30: Standard deviation of elapsed time for the first test set

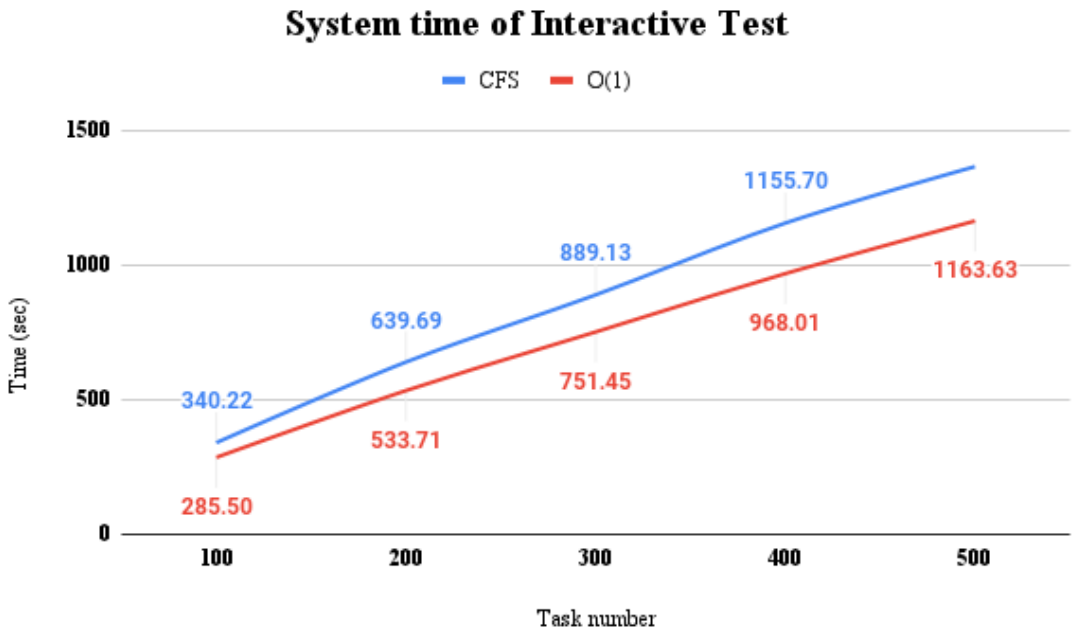


Figure 4.31: Average system time for the first test set

Lastly, as far as involuntary context switches are concerned, the results remain actually the same as in the previous test sets. Their value remains steady as the number of tasks increases, while in CFS, much more context switches occur, than in O(1).

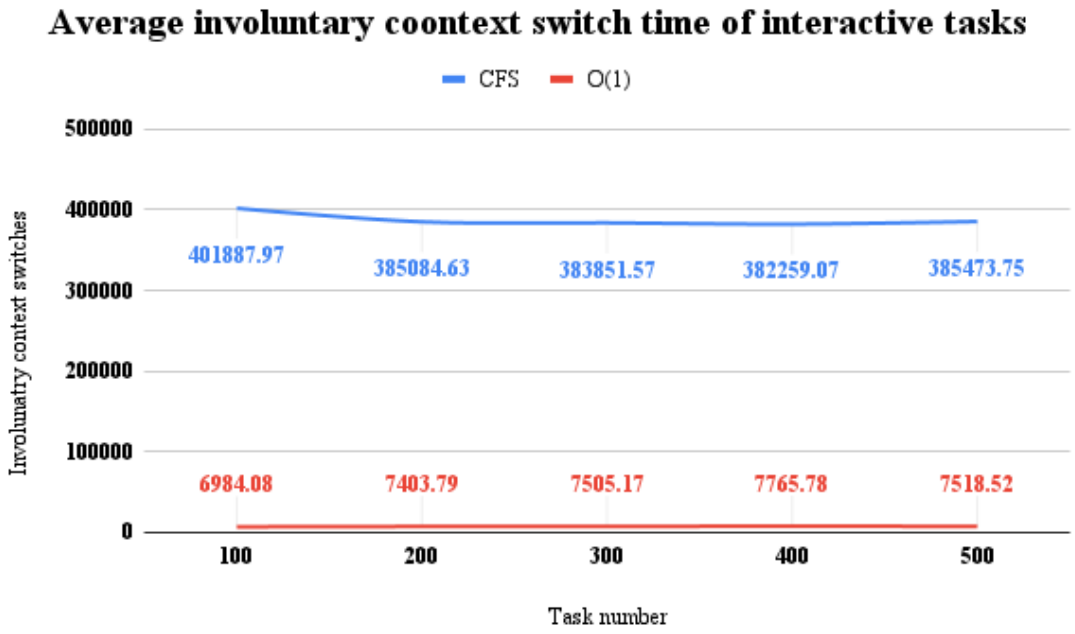


Figure 4.32: Average involuntary context switches for the first test set

### 50 percent interactive tasks

Now half of the tasks are interactive and the other half are batch tasks. The results match the ones from the previous test set. There will be an increase, therefore, in the average values, but the conclusions that can be drawn from these results, in overall, remain the same.

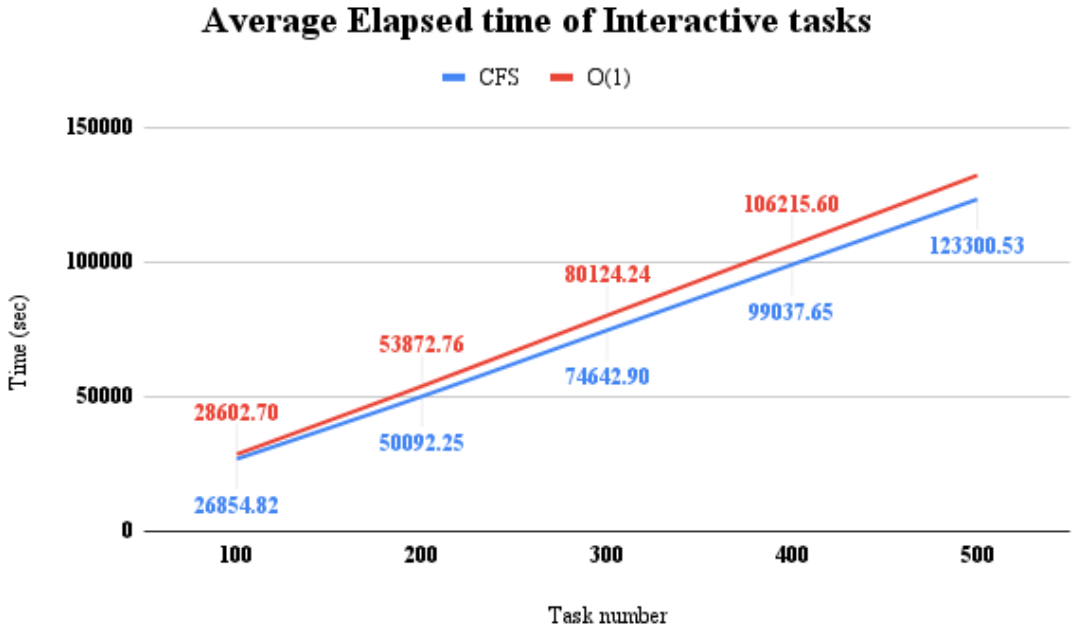


Figure 4.33: Average elapsed time for the second test set

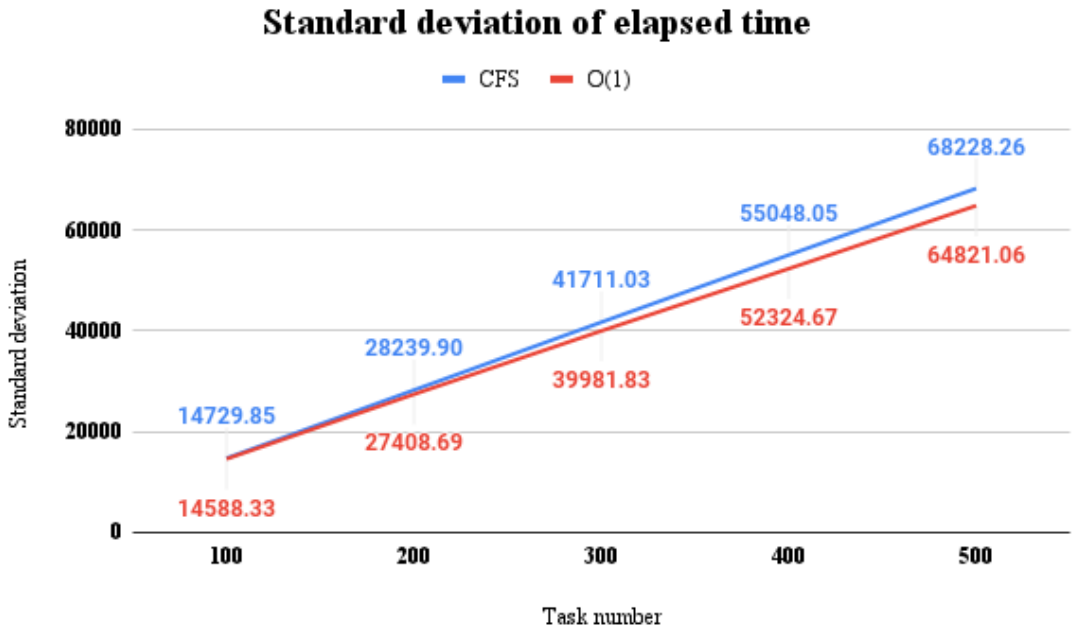


Figure 4.34: Standard deviation of elapsed time for the second test set

What slightly changes, on the other hand, is their fairness. Figure 4.34, explains, that the incrementation of the interactive task percentage, has lead to the faster rise of the difference, between the standard deviation of O(1) and CFS, as the number of total tasks increases. This makes O(1) slightly fairer than CFS.

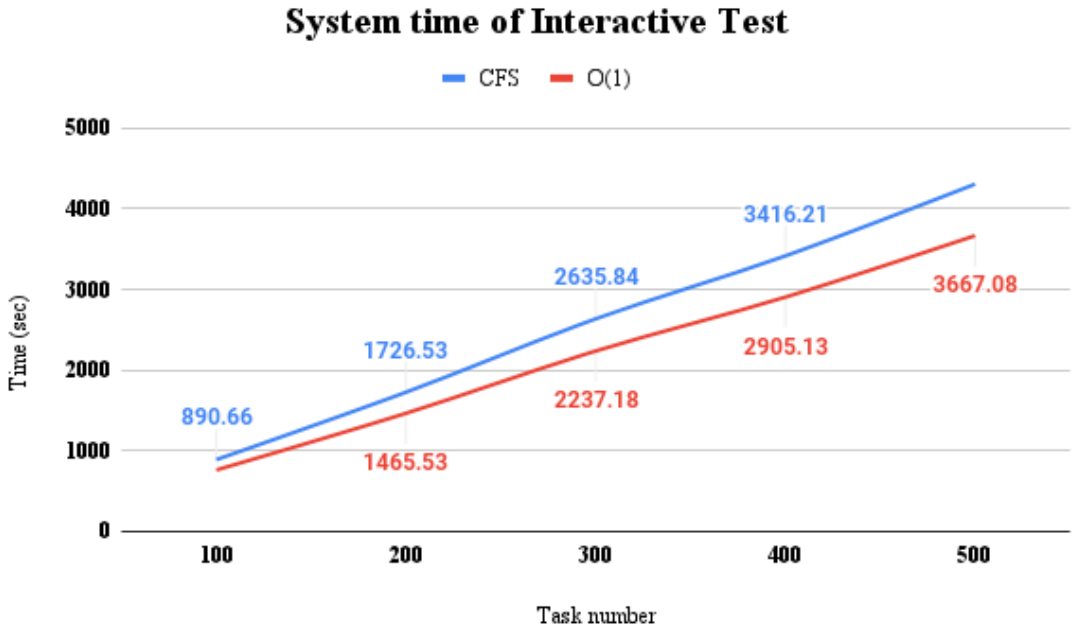


Figure 4.35: Average system time for the second test set

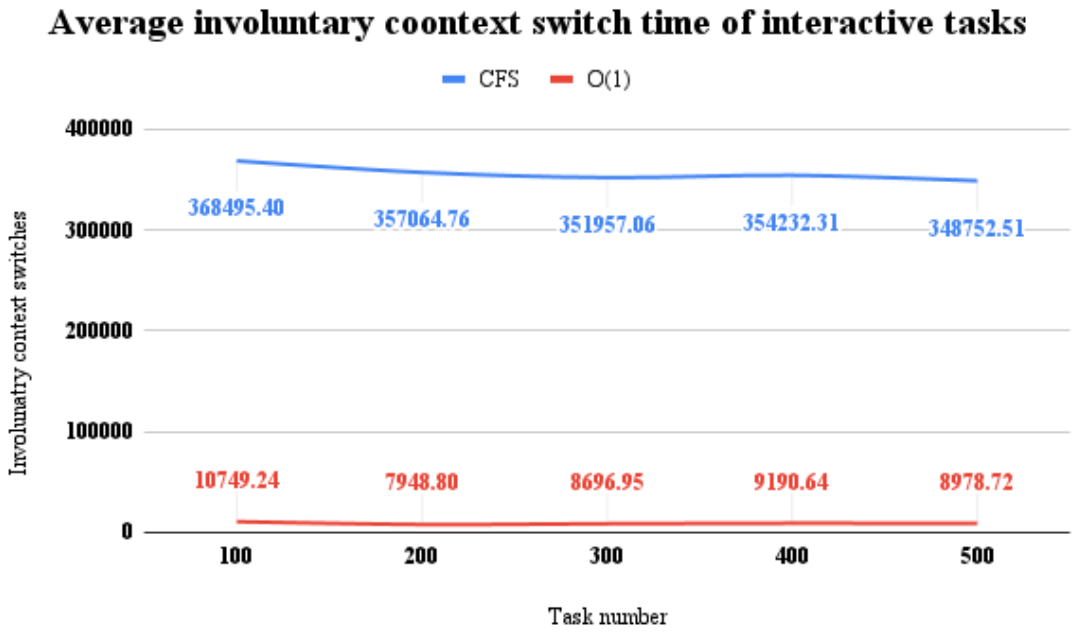


Figure 4.36: Average involuntary context switches for the second test set

### 80 percent interactive tasks

What happens, now, when interactive tasks, are the majority? Taking into consideration figure 4.37, the differences between the elapsed times of O(1) and CFS increases in favor of CFS, as the number of total tasks increases. Also from figure 4.38, we can conclude that they are equal, when measured by their fairness and that their fairness decreases, as the number of tasks increases.

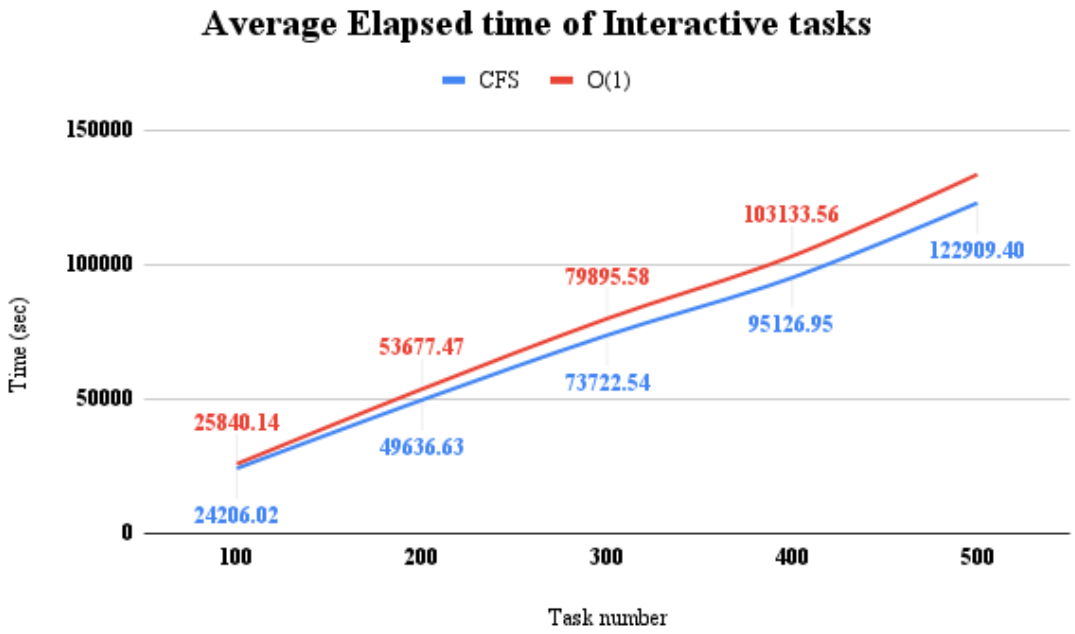


Figure 4.37: Average elapsed time for the third test set

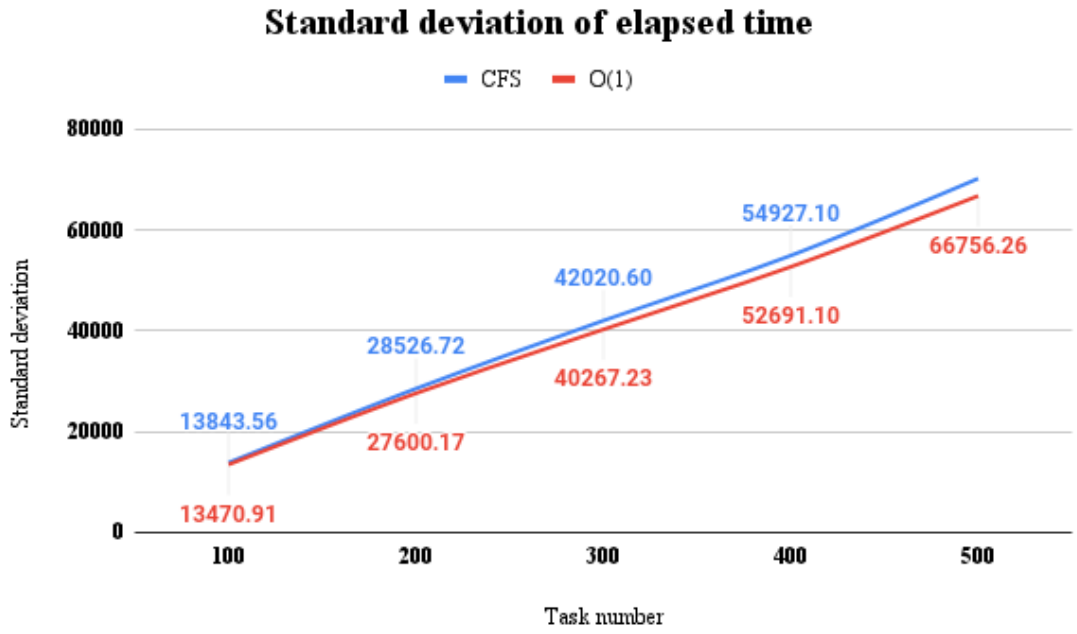


Figure 4.38: Standard deviation of elapsed time for the third test set

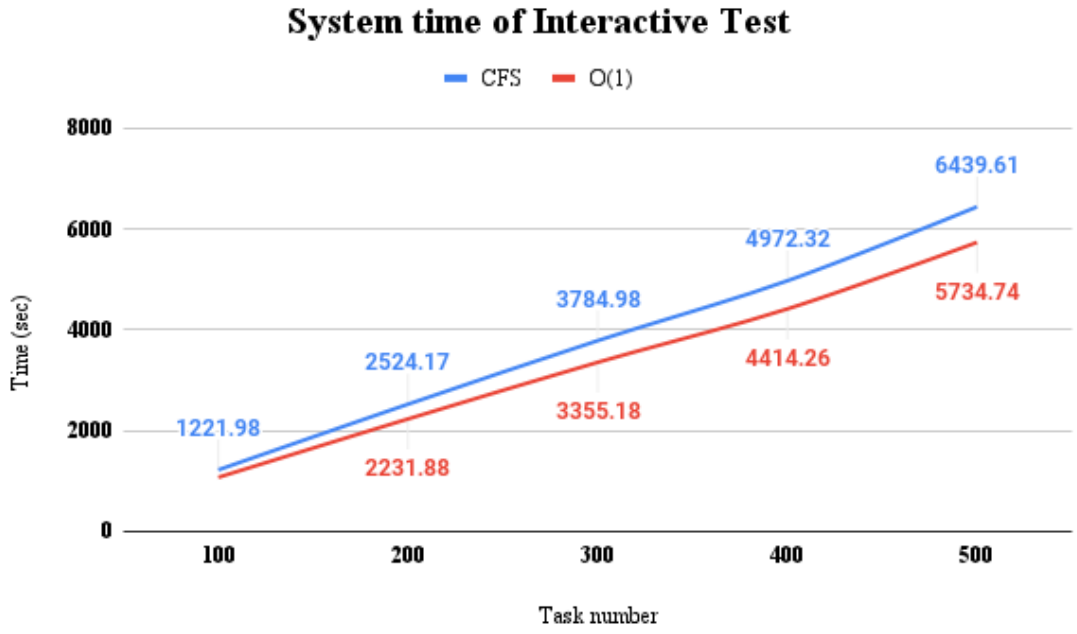


Figure 4.39: Average system time for the third test set

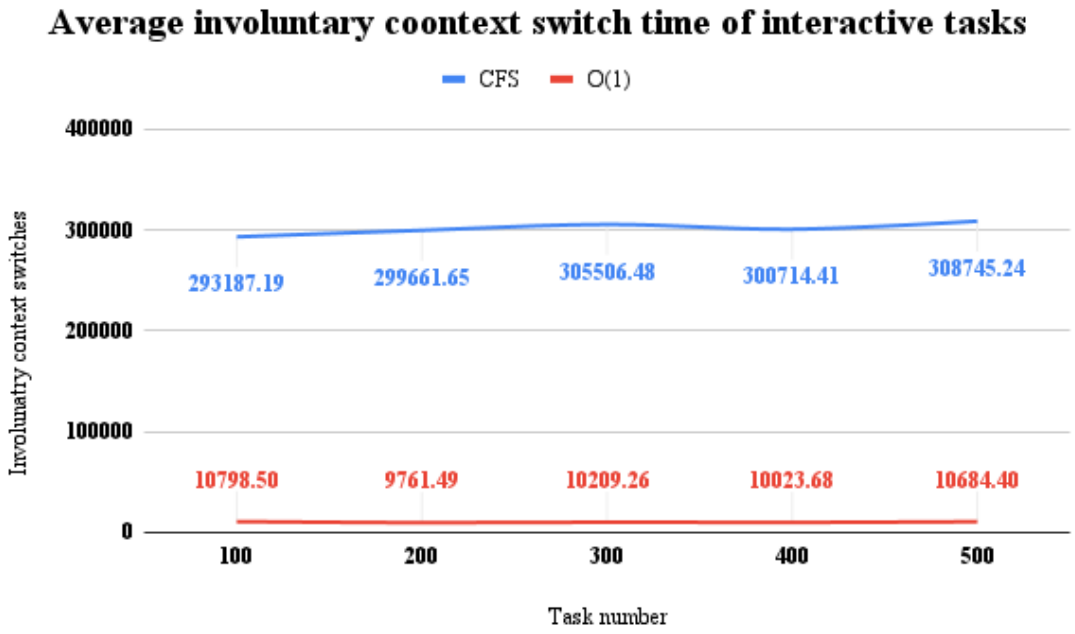


Figure 4.40: Average involuntary context switches for the third test set

#### The conclusion

All in all, keeping the number of interrupts that an interactive task can receive, to a small value and increasing the percentage of the interactive tasks, does not have a huge impact on any of the two schedulers. For now, it seems, the two schedulers behave normally.

### 4.3.2 Mixed tasks: huge amount of interrupts

In the previous set of test sets, we concluded that, there are not many differences, as far as fairness is concerned between the two schedulers, when the amount of interrupts is low and the percentage of interactive tasks increases. Now the amount of interrupts will radically be increased. The results gathered from this set of test sets, are quite interesting.

#### 20 percent interactive tasks

Keeping the interactive task percentage as low as 20 percent of the total tasks, the differences between O(1) and CFS become more clear. First of all, not only did the average elapsed time of O(1) have a higher value than that of CFS, but after the second test, it kept increasing dramatically. Comparing it to the incrementation occurred, when the same tasks are scheduled by CFS, this does not seem like an incrementation at all, on contrast it remains fairly steady.

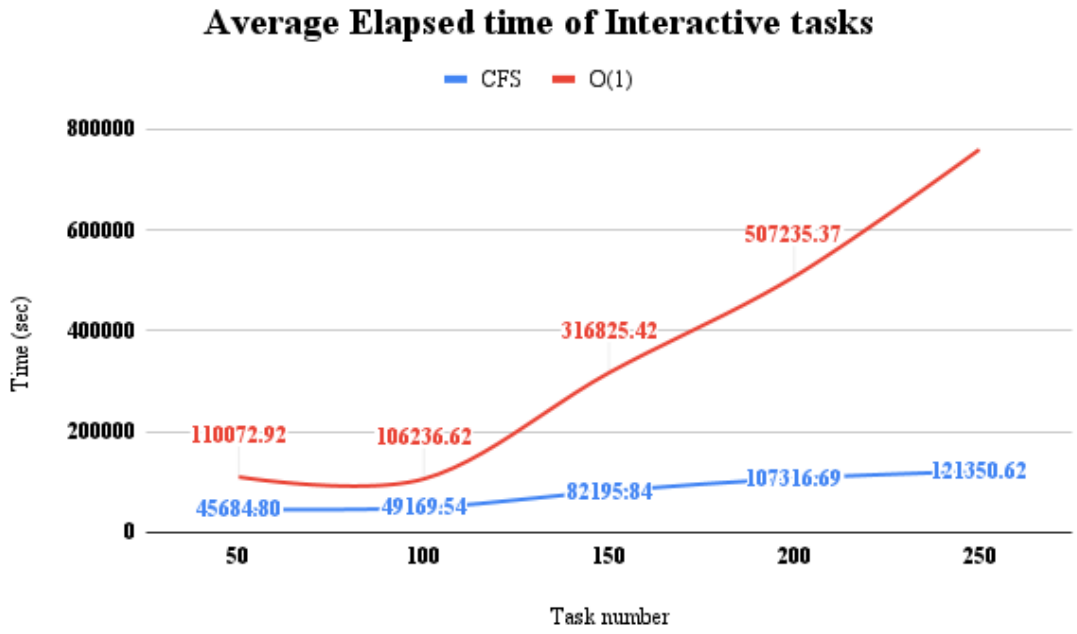


Figure 4.41: Average elapsed time for the first test set

Furthermore, as shown from figure 4.42, the same as before can be said for the standard deviation. CFS is ultimately, fairer than O(1) and manages to keep this fairness, as the number of tasks increases. On the other hand, O(1)'s fairness, radically decreases after the second test, while at the first two tests, it remained steady, but higher than that of CFS.

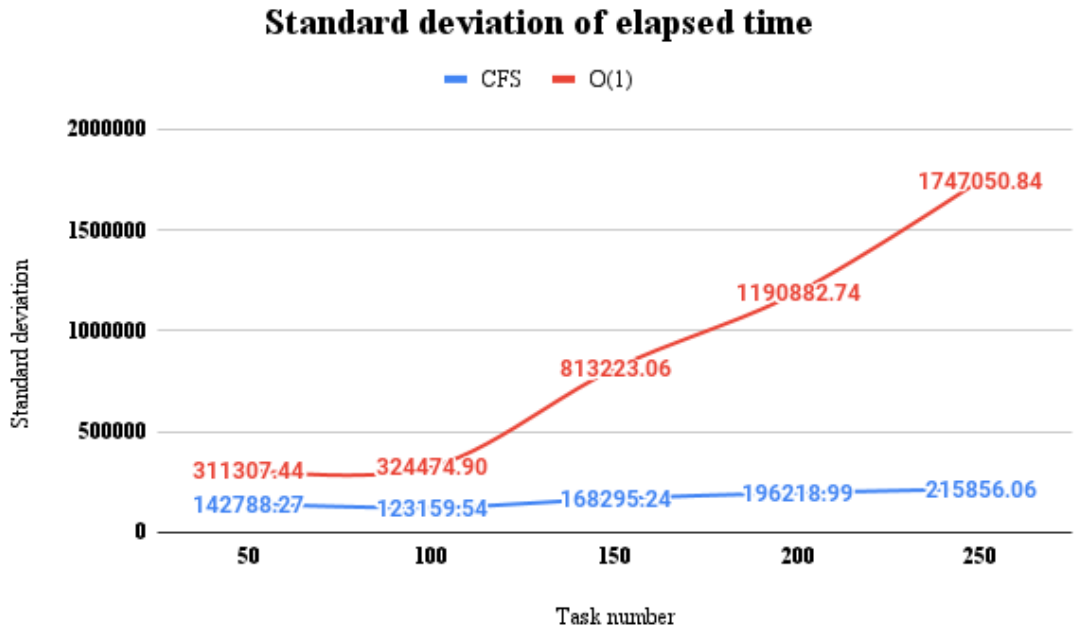


Figure 4.42: Standard deviation of elapsed time for the first test set

CFS still lacks, behind O(1) when comparing them on system time and the

involuntary context switches. The interesting thing, here, is that both manage to maintain the average number of involuntary context switches steady, while the number of total tasks increases.

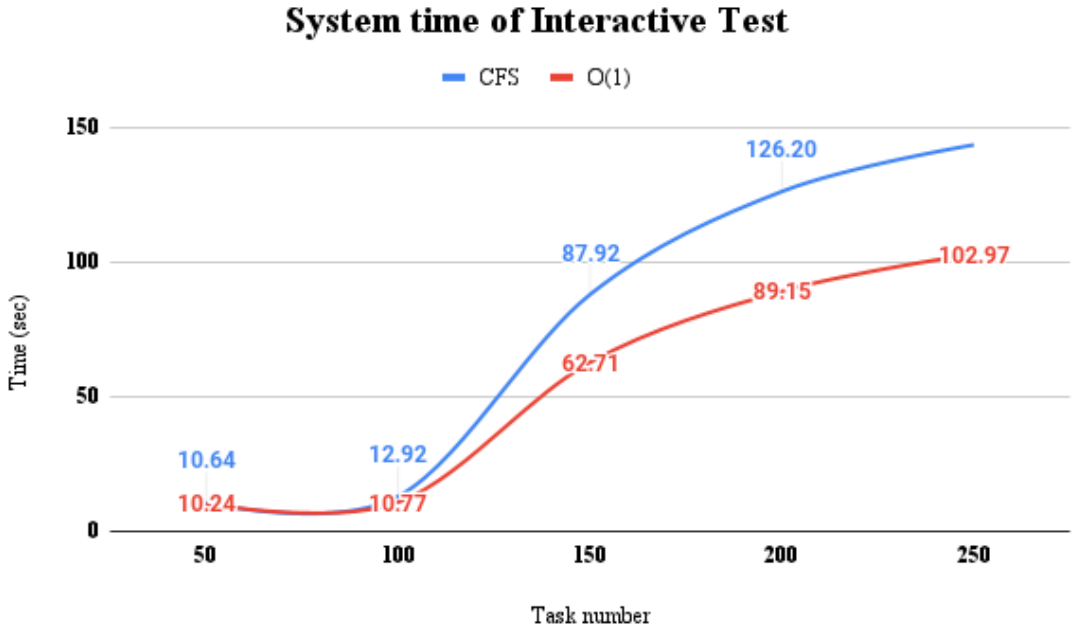


Figure 4.43: Average system time for the first test set

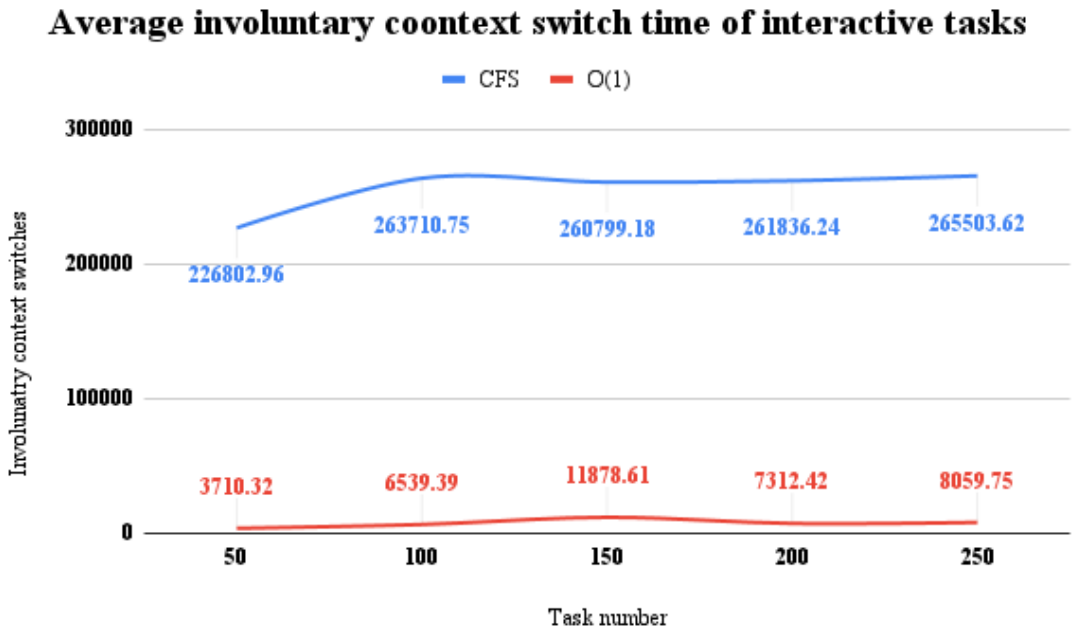


Figure 4.44: Average involuntary context switches for the first test set

### 50 percent interactive tasks

When half of the tasks, are considered interactive, O(1) follows a steady increase of its average elapsed time, as depicted by figure 4.45. On the contrary, CFS manages to keep the average elapsed time steady, while the number of total tasks increases.

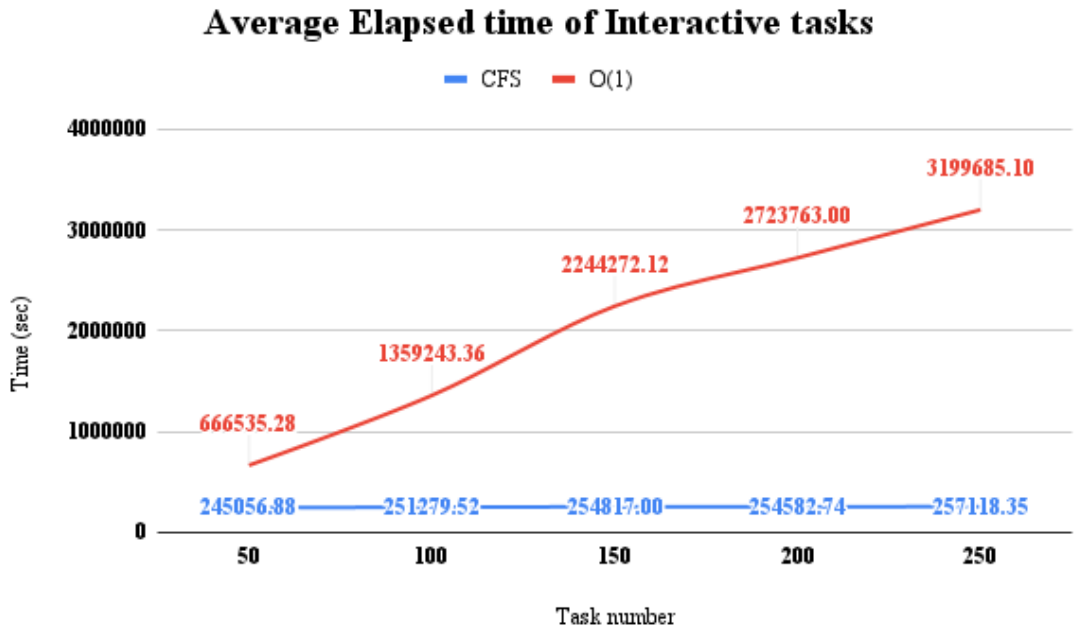


Figure 4.45: Average elapsed time for the second test set

The same thing happens and for the standard deviation of the elapsed times (figure 4.46). This means that the increase of the number of interactive tasks, has lead CFS to becoming more fair and for O(1) to become even less fair, as the number of total tasks increases.

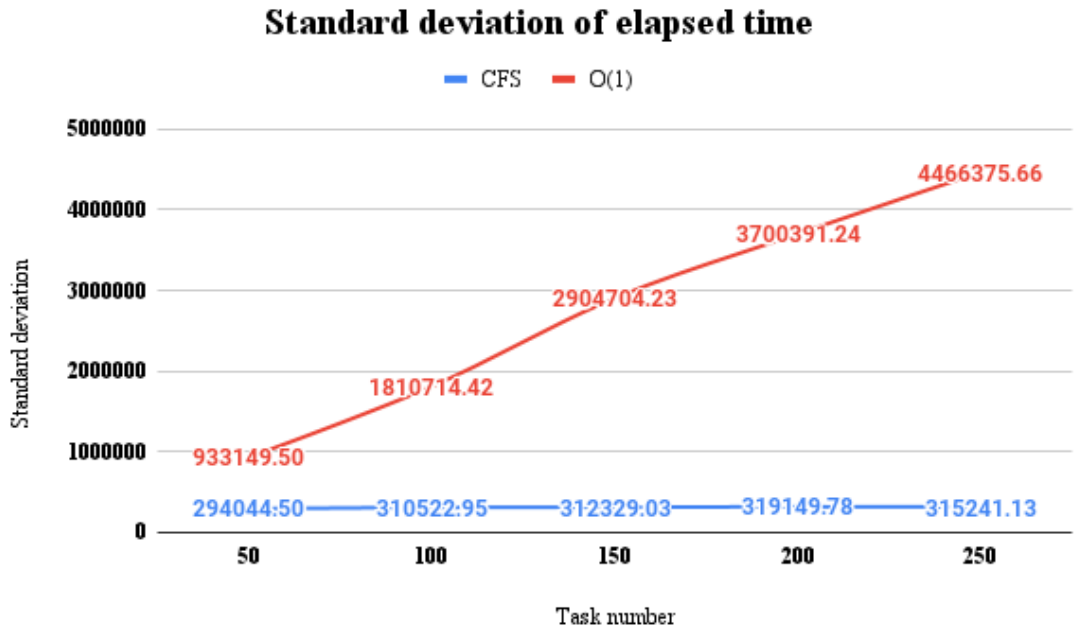


Figure 4.46: Standard deviation of elapsed time for the second test set

Not only this, but CFS manages to achieve average system time values, similar to the ones achieved by O(1). This statement becomes invalid, after the third test.

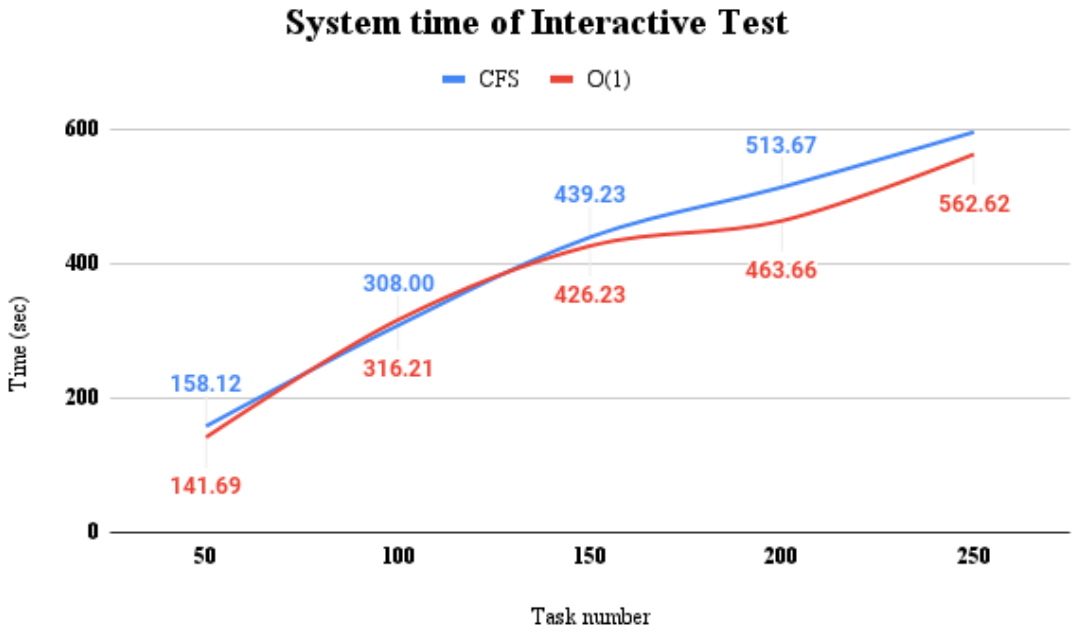


Figure 4.47: Average system time for the second test set

Lastly, for the average involuntary context switches, O(1) as well as CFS, both manage to keep it steady, as the number of tasks increases. For CFS it may have some small, slightly noticeable fluctuations, but theoretically, it remains steady.

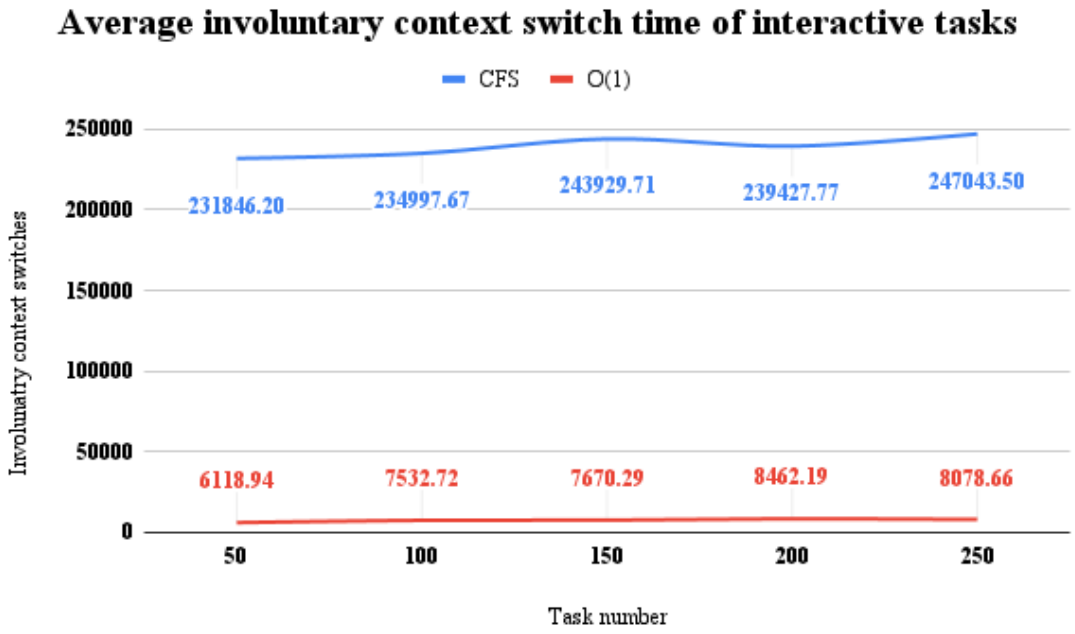


Figure 4.48: Average involuntary context switches for the second test set

### 80 percent interactive tasks

When the majority of the tasks are interactive the difference of the average elapsed time values of O(1) and CFS, becomes larger, even beginning with the first test

(figure 4.49). This value increases faster than in the previous test sets and reaches huge values, while CFS's elapsed time remains similar from one test to the other. Also it is worth noting, that for test number four and five, the average elapsed time for CFS, only slightly changes (test number five has slightly larger values of elapsed time).

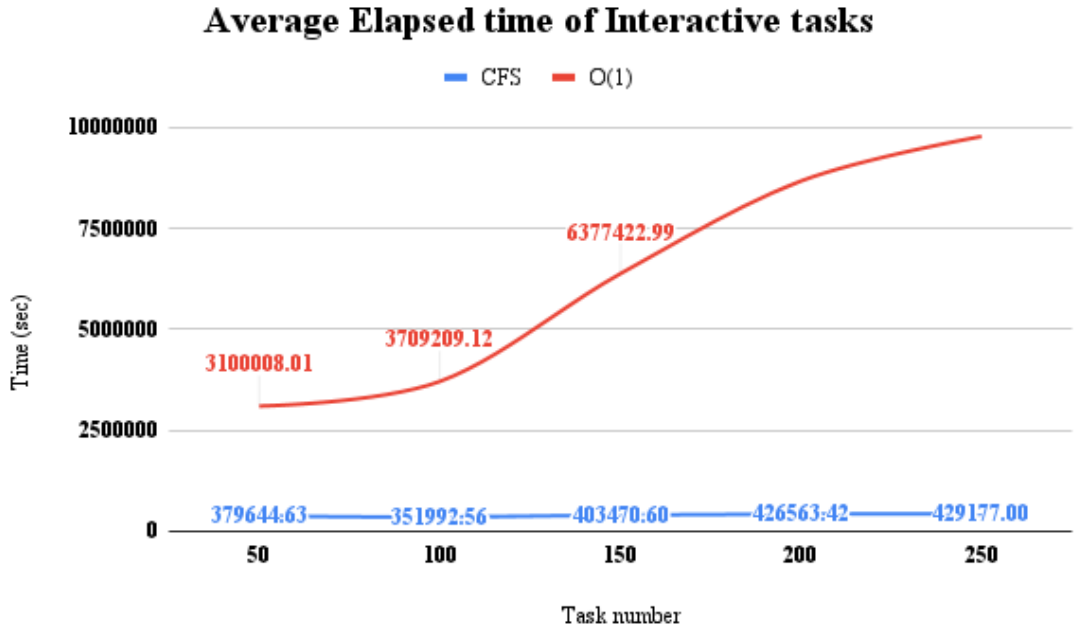


Figure 4.49: Average elapsed time for the third test set

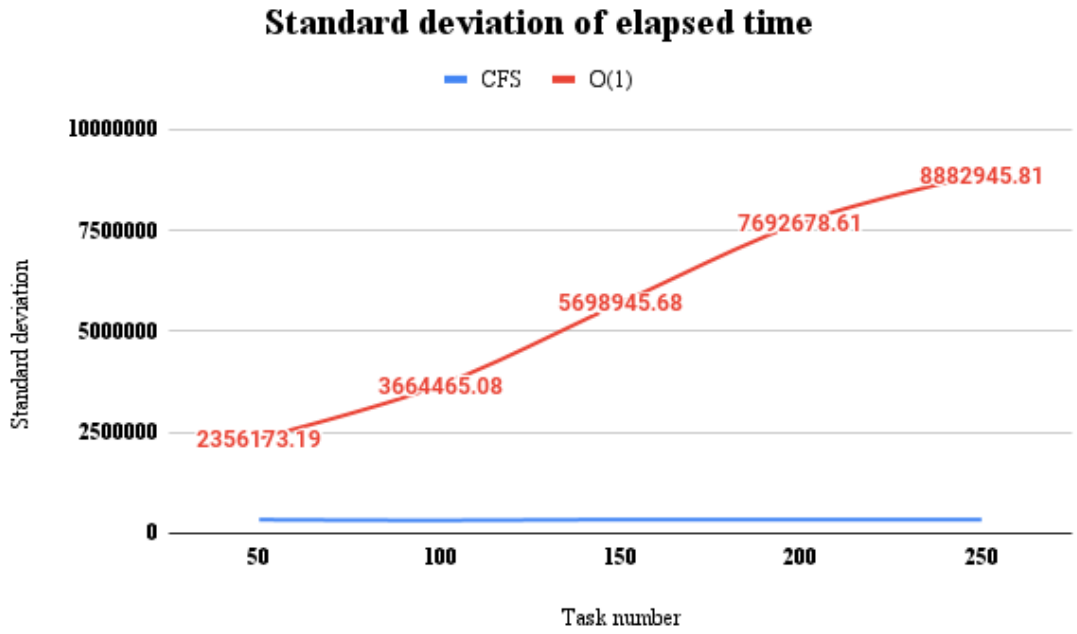


Figure 4.50: Standard deviation of elapsed time for the third test set

As shown in figure 4.50, the gap between CFS's and O(1)'s standard deviation of elapsed time, has increased, when comparing it with the two last test sets. CFS

behaves in the same fair manner with every test conducted, while O(1) becomes less and less fair, while the number of tasks increases.

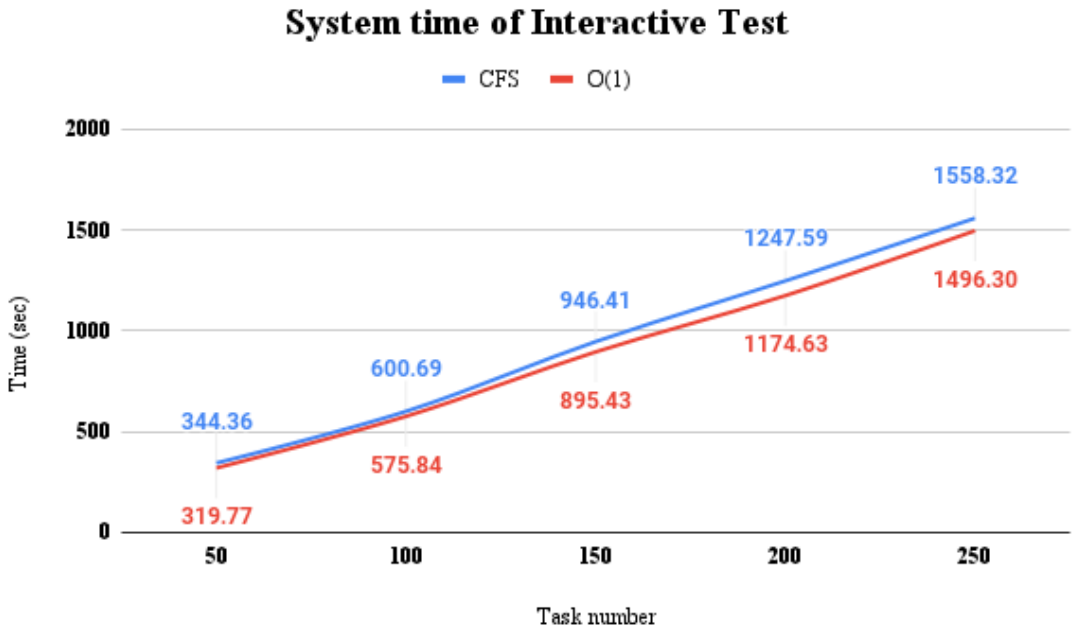


Figure 4.51: Average system time for the third test set

Also, worth noting, is the fact that, the difference of the average system time of the two schedulers, becomes somewhat unnoticeable (figure 4.51).

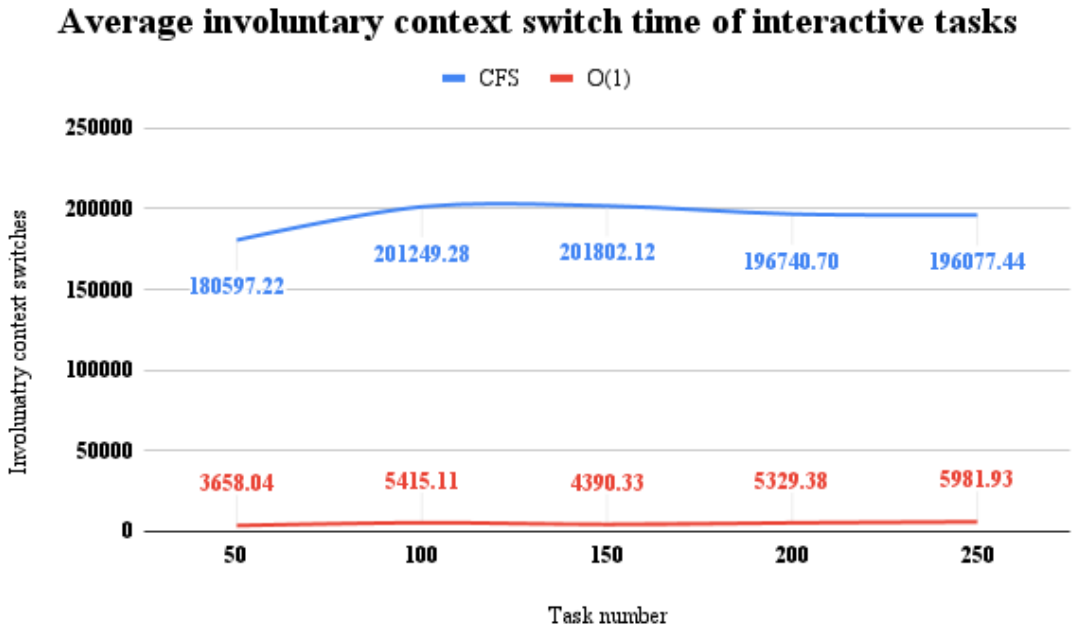


Figure 4.52: Average involuntary context switches for the third test set

The average involuntary context switches occurred by both schedulers are lower than in the previous two test sets. Although, in CFS larger amounts of involuntary

context switches than in O(1), still occur. And as shown from all the previous test sets, this value remains steady, with the increase of the total number of tasks.

### 4.3.3 Summary of the Mixed test

In overall, CFS is better, than O(1) when scheduling mixed tasks. Other than that, it achieves similar system time as O(1) and lower elapsed time, while also maintaining the number of involuntary context switches and thus the time taken to perform them, at very high values. So the assumption made at section 4.2.4, can now be considered as a verified conclusion.

# Chapter 5

# Conclusion and Discussion

## 5.1 Conclusion

The two schedulers have their advantages and disadvantages. Both perform well for fully batch tasks and for interactive tasks with small number of interrupts. Where O(1) seems to become worse in performance than CFS, is when the number of interrupts for the interactive tasks increases and becomes huge. This is because CFS was originally designed, to be implemented on server side ([4]).

Also, as I previously mentioned, O(1) seems to handle CPU monopolization and usage worse than CFS. This assumption is also strengthened by the fact that, in CFS more involuntary context switches occur and also has worse system time than O(1), while having better or similar elapsed time than O(1), which is not preferable on a server machine. But this also means, that CFS wastes many resources on system procedures and context switching, which is not preferable on a desktop machine.

Another important thing, although not the goal of this thesis, but worth noting, is that with the introduction of the scheduling classes in CFS is actually easier to implement new scheduling policies, than it is with the O(1) code (these references, provide some examples of how to implement your own scheduling policy [6] and [7]). This has nothing to do with performance.

All in all, for desktop machines, at least, both schedulers, seem to perform in quite a similar way, with O(1) performing a bit better at times (it actually is more fair). As far as server machines are concerned, that actually depends on the size of the server, but I would say that CFS is better than O(1). So it depends on the workload that the machine is meant to handle, when coming to choose a scheduler, with which the operating system will handle tasks.

## 5.2 Discussion

There are a lot more schedulers, implemented for Linux systems, for example BFS or MuQSS, which were not covered and was not this thesis' goal to cover. Other

than that, a comparison between the schedulers of different operating systems could be interesting.

In anyway, in the future, another scheduler may replace CFS, which may actually be better than both of them. But it may still lack in some points, in which CFS or O(1) may be better at. And that is actually the reason, why, we can not conclude which one is better, because they in some way complete each other. What we can agree on, is that the scheduler is the most fundamental part of an operating system. And it seems that this thesis reached its goal, showing, that the two schedulers complete each other and that there is no such thing such as a better scheduler than another generally, but a better suited one for different workloads.

# References


[1] D. Bovet and M. Cesati, *Understanding The Linux Kernel*. Oreilly & Associates Inc, 2005, ISBN: 0596005652.

[2] W. Mauerer, *Professional Linux Kernel Architecture*. GBR: Wrox Press Ltd., 2008, ISBN: 0470343435.

[3] G. Cheng, "A comparison of two linux schedulers", 2012. [Online]. Available: https://api.semanticscholar.org/CorpusID:59031678.

[4] N. Ishkov, "A complete guide to linux process scheduling", 2015. [Online]. Available: https://api.semanticscholar.org/CorpusID:60292120.

[5] D. Shakoori Gustafsson, "Linux cpu schedulers: Cfs and muqss comparison", ser. UMNAD 1289, 2021, p. 18.

[6] P. Baltarejo and L. L. Ferreira, *Implementing a new real-time scheduling policy for linux: Part 1*, https://www.embedded.com/implementing-a-new-real-time-scheduling-policy-for-linux-part-1/, Checked: 25/11/2023.

[7] P. Baltarejo and L. L. Ferreira, *Implementing a new real-time scheduling policy for linux: Part 2*, https://www.embedded.com/implementing-a-new-real-time-scheduling-policy-for-linux-part-2/, Checked: 25/11/2023.

[8] *Operating systems: Cpu scheduling*, https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/6_CPU_Scheduling.html, Checked: 19/10/2023.

[9] D. Porter, *Scheduling processes*, https://www.cs.unc.edu/~porter/courses/comp530/f16/slides/scheduling-intro.pdf, Checked: 19/10/2023.

[10] L. Torvalds, *Linux kernel code, from bootlin*, https://elixir.bootlin.com/linux/v2.6.0/source, Checked: 2/11/2023.

[11] L. Torvalds, *Linux kernel code, from bootlin*, https://elixir.bootlin.com/linux/v2.6.24/source, Checked: 2/11/2023.